

LAPORAN BASIS DATA KELOMPOK G



Disusun untuk memenuhi tugas terstruktur :

Mata kuliah : Basis Data

Dosen : Lutfiah Maharani Siniwi, M.Stat.

DISUSUN OLEH:

- | | |
|---------------------------|-------------|
| 1. Salman Alfarisy | (K1D024010) |
| 2. Muhammad Rafi Abdillah | (K1D024018) |
| 3. Faricha Cahya Metia | (K1D024023) |
| 4. Arya Daffa Pratama | (K1D024033) |
| 5. Fatihaturrohman | (K1D024035) |

KEMENTERIAN PENDIDIKAN TINGGI, SAINS, DAN TEKNOLOGI

UNIVERSITAS JENDERAL SOEDIRMAN

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

PROGRAM STUDI STATISTIKA

PURWOKERTO

2026

DAFTAR ISI

BAB I

PENDAHULUAN.....	13
1.1. Latar Belakang.....	13
1.2. Rumusan Masalah.....	14
1.3. Tujuan.....	15
1.4. Manfaat Penelitian.....	15

BAB II

ANALISIS KEBUTUHAN.....	16
2.1. Deskripsi studi Kasus dan Ruang Lingkup Sistem.....	16
2.2. Identifikasi Aktor (Pengguna Sistem).....	16
2.3. Daftar Kebutuhan Fungsional Terkait Data.....	17
2.4. Aturan Bisnis (Business Rules).....	18

BAB III

PERANCANGAN BASIS DATA.....	20
3.1. Alur Pengerjaan.....	20
3.2. Entity Relationship Diagram (ERD).....	21
3.3. Deskripsi Entitas dan Atribut.....	22
3.4. Kardinalitas Relasi Antar Entitas.....	28
3.5. Normalisasi Data.....	30
3.6. Kamus Data (Data Dictionary).....	35

BAB IV

IMPLEMENTASI.....	42
4.1. Implementasi Database.....	42
4.2. Hasil Implementasi Database.....	70
4.3. Relasi Antar Tabel.....	71
4.4. Ringkasan Implementasi.....	72

BAB V

PENGUJIAN QUERY.....	74
5.1. Pengujian Query SELECT (10 Query Bertingkat).....	74

5.2. Pengujian Logical View (2 Buah VIEW).....	86
5.3. Pengujian Stored Procedure (1 Buah).....	89
5.4. Pengujian Database Trigger (1 Buah).....	90
5.5. Pengujian User-Defined Function (1 Buah).....	92
BAB VI	
KESIMPULAN DAN SARAN.....	95
6.1. Kesimpulan.....	95
6.2. Saran.....	97
LAMPIRAN.....	100

BAB I

PENDAHULUAN

1.1. Latar Belakang

Rekayasa teknologi informasi dan komunikasi yang berkembang pesat dalam beberapa dekade terakhir telah menstimulasi transformasi masif pada berbagai sektor, termasuk reorganisasi sistem perdagangan konvensional. Manifestasi paling signifikan dari digitalisasi ini terlihat dari akselerasi industri *e-commerce* di Indonesia, di mana berbagai platform digital tumbuh dan saling berkompetisi secara ketat untuk menyajikan kapabilitas layanan optimal bagi pengguna. Implikasi dari penetrasi teknologi ini secara fundamental telah merekonstruksi perilaku konsumsi masyarakat dalam bertransaksi. Kehadiran platform perdagangan elektronik mereduksi hambatan geografis dengan memfasilitasi aktivitas jual beli secara daring, sehingga memberikan efisiensi tinggi bagi konsumen untuk berbelanja tanpa ketergantungan fisik pada gerai retail atau pusat perbelanjaan.

Secara konseptual, *e-commerce* merupakan suatu ekosistem digital untuk saling menukar barang atau jasa guna memenuhi kebutuhan operasional maupun sehari-hari. Seiring dengan perkembangan ilmu pengetahuan tersebut, sistem transaksi secara *online* ini memberikan kemudahan besar bagi para pelaku bisnis maupun pengelola platform untuk saling berinteraksi, mengelola pasar, serta bertransaksi secara instan melalui media internet. Pergeseran pola transaksi yang semakin dinamis ini pada akhirnya menuntut adanya sistem pendukung (*backend*) yang tidak hanya adaptif, melainkan juga memiliki tingkat skalabilitas tinggi guna menyelaraskan arus lalu lintas data yang terus melonjak. Atas dasar kebutuhan tersebut, operasional platform *marketplace* yang ideal wajib ditopang oleh rancangan arsitektur basis data yang baik guna mendukung efektivitas pemrosesan transaksi sekaligus pendokumentasian informasi secara sistematis. Implementasi basis data relasional menjadi pilihan utama untuk menggantikan metodologi pengelolaan informasi konvensional yang masih bersifat manual. Basis data memiliki kapabilitas sebagai media penyimpanan terpusat yang mampu menampung volume data dalam jumlah besar, memangkas waktu pengaksesan data agar lebih cepat, serta menyederhanakan kontrol data operasional seperti proses penambahan, pengambilan, penyimpanan, hingga modifikasi informasi secara aman dan efisien. Pada praktiknya, struktur basis

data yang kokoh inilah yang nantinya akan mengintegrasikan seluruh entitas data kompleks di dalam *marketplace*, mulai dari data pelanggan, produk, transaksi pembayaran, hingga ulasan konsumen, sekaligus melindunginya dari risiko anomali atau penumpukan data ganda (*redundansi*).

Berdasarkan kompleksitas tata kelola data yang terjadi pada industri pasar digital tersebut, proyek akhir mata kuliah Basis Data ini difokuskan pada perancangan dan implementasi sistem basis data relasional yang utuh untuk studi kasus *Marketplace/ E-commerce*. Pemilihan topik ini didasarkan pada kebutuhan nyata untuk mensimulasikan sebuah ekosistem bisnis daring yang mengelola berbagai variasi kategori produk, seperti *fashion*, kuliner, produk kecantikan, dan lain sebagainya, yang memiliki alur perputaran stok sangat dinamis. Perancangan ini menerapkan prinsip arsitektur data modern, mulai dari pemodelan konseptual menggunakan *Entity Relationship Diagram* (ERD), pengujian dependensi fungsional melalui proses normalisasi hingga bentuk normal ketiga (3NF) guna mengeliminasi anomali, hingga pemrograman objek basis data tingkat lanjut (*stored procedure, function, view, dan trigger*). Dengan implementasi tersebut, diharapkan sistem mampu melakukan otomatisasi operasional seperti pemotongan stok barang secara otomatis di berbagai kategori produk saat pembayaran terkonfirmasi serta menyajikan integritas data yang tinggi bagi keberlangsungan bisnis platform.

1.2. Rumusan Masalah

Berdasarkan latar belakang yang telah dipaparkan, maka rumusan masalah dalam proyek perancangan basis data *marketplace* ini adalah:

1. Bagaimana mengidentifikasi kebutuhan data dan merancang struktur tabel (*skema relasional*) yang optimal guna mengelola berbagai variasi kategori produk secara terintegrasi?
2. Bagaimana menerapkan proses normalisasi hingga bentuk *Third Normal Form* (3NF) pada data transaksi *marketplace* untuk mengeliminasi risiko redundansi dan anomali data?
3. Bagaimana mengimplementasikan objek basis data tingkat lanjut (*stored procedure, function, view, dan trigger*) pada DBMS relasional untuk mendukung otomatisasi operasional bisnis seperti sinkronisasi pemotongan stok barang sekaligus menjaga integritas data platform?

1.3. Tujuan

Tujuan yang ingin dicapai dari pelaksanaan proyek akhir ini adalah:

1. Mampu menganalisis kebutuhan sistem informasi *marketplace* dan memetakannya ke dalam skema *Entity Relationship Diagram* (ERD) konseptual dan logikal untuk manajemen produk.
2. Mampu menyusun struktur tabel yang bersih dari anomali data melalui tahapan normalisasi data yang sistematis dari bentuk unnormalized, 1NF, 2NF, hingga mencapai 3NF.
3. Mampu mengimplementasikan logika internal database menggunakan *query* kompleks serta fitur pemrograman lanjutan (*trigger, procedure, view, function*) yang berjalan secara benar, otomatis, dan efisien pada DBMS.

1.4. Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat sebagai berikut:

1. Memberikan pemahaman dan keterampilan dalam menganalisis kebutuhan sistem informasi marketplace serta merancang basis data menggunakan Entity Relationship Diagram (ERD) konseptual dan logikal.
2. Meningkatkan kemampuan dalam menyusun struktur tabel yang terorganisir melalui proses normalisasi data dari bentuk unnormalized hingga Third Normal Form (3NF) sehingga dapat mengurangi redundansi dan anomali data.
3. Melatih kemampuan dalam mengimplementasikan logika internal database menggunakan query kompleks serta fitur pemrograman lanjutan seperti trigger, procedure, view, dan function untuk mendukung pengelolaan data yang lebih efisien dan otomatis.

BAB II

ANALISIS KEBUTUHAN

2.1. Deskripsi studi Kasus dan Ruang Lingkup Sistem

Project ini berfokus pada perancangan dan implementasi sistem basis data relasional untuk platform *E-Commerce / Marketplace* skala menengah. Studi kasus yang diambil adalah sistem operasional toko daring (*online marketplace*) yang memfasilitasi transaksi jual-beli produk secara elektronik. Sistem basis data yang dirancang bertindak sebagai mesin utama (*back-end data store*) untuk mengelola seluruh siklus data operasional, mulai dari manajemen akun pengguna, pengelolaan katalog produk oleh penjual, proses pemesanan barang oleh pelanggan, sampai pencatatan transaksi pembayaran.

Ruang lingkup dari sistem basis data *e-commerce* ini dibatasi pada fungsionalitas inti berikut:

1. Manajemen Pengguna & Alamat: Mencatat data profil pelanggan dan memungkinkan satu pelanggan menyimpan lebih dari satu alamat pengiriman (*one-to-many relationship*).
2. Manajemen Katalog Produk: Mengelola data produk yang terbagi ke dalam kategori-kategori spesifik serta memantau jumlah stok barang secara dinamis.
3. Mekanisme Transaksi Penjualan: Mencatat pembuatan pesanan (*order*), mendeteksi kuantitas item yang dibeli, dan mengunci harga satuan produk pada saat transaksi terjadi melalui tabel detail pesanan.
4. Sistem Pembayaran dan Logistik: Mengintegrasikan status konfirmasi pembayaran serta mencatat informasi pengiriman barang, termasuk pemilihan perusahaan kurir, jenis layanan, dan nomor resi pelacakan.
5. Sistem Ulasan (*Feedback*): Memfasilitasi pencatatan penilaian (*rating*) dan ulasan tekstual dari pelanggan terhadap produk yang telah berhasil dibeli.

2.2. Identifikasi Aktor (Pengguna Sistem)

Untuk menjamin keamanan dan kesesuaian hak akses data, sistem ini mengidentifikasi 3 (tiga) aktor utama yang berinteraksi langsung dengan basis data:

1. Pelanggan (*Customer*)

- a) Deskripsi: Pengguna akhir (*end-user*) yang menggunakan *platform* untuk mencari produk, melakukan pemesanan, dan melakukan pembayaran.
 - b) Hak Akses Data: Memiliki hak akses penuh untuk membaca (*SELECT*) katalog produk, menambah/mengubah (*INSERT/UPDATE*) data profil dan alamat pribadi, membuat (*INSERT*) data pesanan baru, serta memberikan (*INSERT*) ulasan produk. Pelanggan tidak dapat melihat data sensitif pelanggan lain maupun mengubah data stok produk secara langsung.
2. Manajer Toko / Administrator Sistem (Admin)
- a) Deskripsi: Aktor internal yang bertanggung jawab atas manajemen operasional *marketplace*, pemeliharaan data master, dan pemantauan transaksi.
 - b) Hak Akses Data: Memiliki hak akses menyeluruh (*privilege* penuh: *SELECT*, *INSERT*, *UPDATE*, *DELETE*) terhadap data master kategori, produk, kurir, dan metode pembayaran. Admin juga memiliki hak untuk memantau seluruh riwayat pesanan, memproses laporan penjualan, serta mengelola status ulasan pengguna.
3. Kurir / Pihak Logistik (*Courier*)
- a) Deskripsi: Aktor eksternal atau sistem kemitraan pihak ketiga yang bertanggung jawab terhadap pembaruan status distribusi barang.
 - b) Hak Akses Data: Memiliki hak akses terbatas untuk membaca (*SELECT*) data alamat pengiriman pelanggan dan detail pesanan yang harus dikirim, serta hak untuk mengubah (*UPDATE*) status pengiriman dan menginput nomor resi pada tabel pengiriman.

2.3. Daftar Kebutuhan Fungsional Terkait Data

Kebutuhan fungsional data memetakan informasi apa saja yang wajib disimpan, diolah, dan disajikan oleh sistem basis data untuk mendukung proses bisnis *e-commerce*:

- 1. Kebutuhan Data Pengguna & Autentikasi:
 - a) Sistem harus mampu menyimpan data identitas unik pelanggan (ID Pelanggan, Nama Lengkap, Email, Nomor HP, dan Tanggal Daftar).
 - b) Sistem harus mampu menyimpan data multi-alamat untuk setiap pelanggan (Alamat Lengkap, Kota, Provinsi, dan Kode Pos).

2. Kebutuhan Data Katalog Produk:
 - a) Sistem harus mampu mengelompokkan produk ke dalam kategori tertentu.
 - b) Sistem harus mencatat informasi mendetail mengenai produk (Nama Produk, Deskripsi, Harga Jual, dan Jumlah Stok yang tersedia).
3. Kebutuhan Data Transaksi & Pesanan:
 - a) Sistem harus mampu mencatat setiap transaksi pemesanan yang dibuat oleh pelanggan, lengkap dengan stempel waktu (*timestamp*) transaksi dan total harga bruto.
 - b) Sistem harus mampu merekam detail item yang dibeli dalam satu pesanan (ID Produk, jumlah kuantitas, dan harga satuan riil saat dibeli).
4. Kebutuhan Data Pembayaran & Pengiriman:
 - a) Sistem harus mencatat status pembayaran (Belum Bayar, Menunggu Konfirmasi, Lunas) beserta metode pembayaran yang dipilih.
 - b) Sistem harus menyimpan data logistik pengiriman (Nama Kurir, Nomor Resi, Tanggal Kirim, dan Status Pengiriman seperti 'Diproses', 'Dikirim', 'Selesai').
5. Kebutuhan Pelaporan & Analisis:

Sistem harus mampu menyajikan data statistik penjualan, produk terlaris, dan ulasan kepuasan pelanggan secara berkala melalui objek kueri dan *view*.

2.4. Aturan Bisnis (*Business Rules*)

Business rules menetapkan batasan operasional, batasan struktural, dan validasi data yang wajib ditegakkan di dalam skema basis data melalui penggunaan *constraints* (PK, FK, CHECK, UNIQUE, NOT NULL) maupun objek lanjutan seperti *trigger*. Berikut adalah aturan bisnis sistem *e-commerce* ini:

1. Aturan Akun & Pelanggan:
 - a) Setiap pelanggan diidentifikasi dengan ID Pelanggan yang unik (*Primary Key*).
 - b) Satu alamat surel (*email*) hanya dapat digunakan oleh satu akun pelanggan (*Constraint UNIQUE*).
 - c) Seorang pelanggan dapat mendaftarkan banyak alamat pengiriman, namun setiap alamat pengiriman hanya boleh merujuk ke satu pelanggan pemilik (*One-to-Many*).
2. Aturan Produk & Stok:

- a) Setiap produk wajib dikategorikan ke dalam minimal satu kategori produk yang valid (*Constraint Foreign Key*).
- b) Harga produk dan jumlah stok tidak boleh bernilai negatif (≥ 0) (*Constraint CHECK*).
- c) Jika stok suatu produk bernilai 0, maka produk tersebut tidak dapat dimasukkan ke dalam keranjang belanja atau pesanan baru.

1. Aturan Transaksi & Pembelian:

- a) Satu pesanan dapat terdiri atas satu atau beberapa produk berbeda. Sebaliknya, satu jenis produk dapat dipesan dalam banyak transaksi pesanan yang berbeda (*Many-to-Many*, didekomposisi melalui tabel Detail Pesanan).
- b) Harga produk yang tercatat pada tabel Detail Pesanan harus mengunci harga riil produk pada saat transaksi dilakukan, sehingga jika di kemudian hari Admin mengubah harga produk pada tabel utama, nilai transaksi masa lalu tidak ikut berubah.

2. Aturan Pembayaran & Pengiriman:

- a) Transaksi pesanan baru akan diproses ke tahap pengiriman barang jika dan hanya jika status pembayaran pada tabel Pembayaran telah dikonfirmasi "Lunas" oleh sistem/admin.
- b) Nomor resi pengiriman bersifat unik (*Constraint UNIQUE*) dan hanya boleh diinput oleh aktor Kurir setelah barang diserahkan ke pihak logistik.

3. Aturan Otomatisasi (*Trigger Logic*):

- a) Setiap terjadi transaksi pembelian baru (operasi *INSERT* pada tabel Detail Pesanan), basis data harus secara otomatis memotong jumlah stok produk pada tabel Produk dalam kuantitas yang sesuai (*Automated Trigger*).
- b) Pelanggan hanya diperbolehkan memberikan ulasan (*INSERT* pada tabel Ulasan) untuk produk yang memiliki riwayat status transaksi "Selesai" pada tabel Pesanan terkait.

BAB III

PERANCANGAN BASIS DATA

3.1. Alur Pengerjaan

Dalam membangun sistem basis data *E-Commerce* ini, kelompok kami menerapkan alur pengerjaan yang terstruktur dan bertahap. Setiap tahapan dilakukan secara berurutan untuk memastikan integritas data, efisiensi struktur tabel, serta berjalannya fungsi logika bisnis sistem dengan baik. Secara mendetail, penjelasan dari setiap tahapan alur pengerjaan proyek basis data kelompok kami adalah sebagai berikut:

1. Merancang Entity Relationship Diagram (ERD)

Langkah awal dimulai dengan merancang ERD secara konseptual. Pada tahap ini, kami mengidentifikasi entitas-entitas master dan transaksi yang diperlukan oleh sistem *e-commerce* (seperti pelanggan, produk, pesanan, dan review) beserta atribut-atribut yang melekat pada masing-masing entitas tersebut.

2. Menghubungkan Primary Key (PK) dan Foreign Key (FK)

Setelah entitas terbentuk, kami melakukan pemetaan skema relasional (*logical design*). Pada tahap ini, kami menentukan secara tegas *Primary Key* (PK) sebagai identitas unik di setiap tabel, lalu menghubungkannya dengan *Foreign Key* (FK) pada tabel relasinya untuk menjaga integritas referensial antar tabel.

3. Merancang dan Membuat Data Dummy

Sebelum mentransfer skema ke dalam *Database Management System* (DBMS), kami menyusun data acuan atau data *dummy* yang realistis. Data ini dirancang sedemikian rupa agar saling sinkron antar tabel relasi, dengan ketentuan setiap tabel utama diisi minimal 20 baris data untuk keperluan pengujian sistem nantinya.

4. Proses Normalisasi Tabel

Untuk menjamin struktur database yang optimal, kami melakukan pengujian normalisasi minimal hingga bentuk normal ketiga (3NF). Tahapan ini bertujuan untuk menghilangkan redudansi (data ganda) dan menghindari

anomali data (*insert, update, delete anomalies*), seperti memisahkan kolom kota dan provinsi dari tabel alamat menjadi tabel wilayah tersendiri.

5. Implementasi Script DDL dan DML

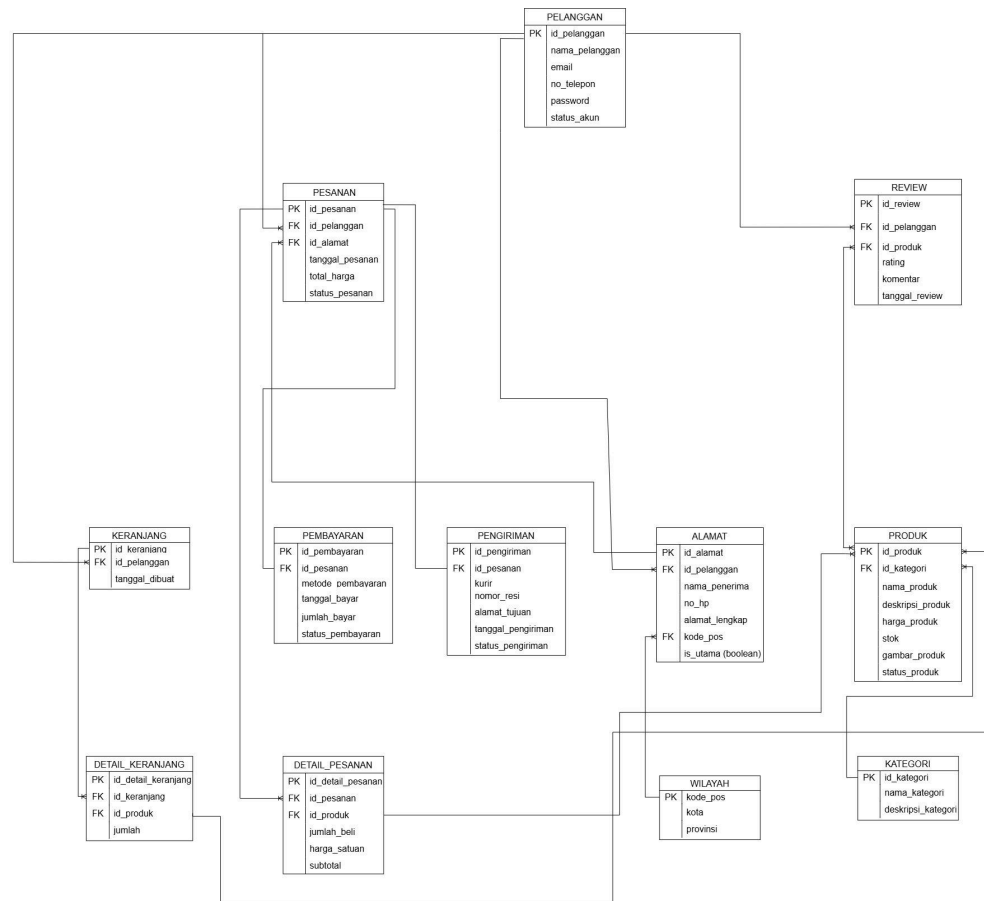
Setelah rancangan tabel dinyatakan legal dan normal, kami mengimplementasikannya ke dalam DBMS MySQL. Langkah ini diawali dengan mengeksekusi script DDL (*Data Definition Language*) berupa perintah CREATE TABLE beserta seluruh *constraint*-nya, kemudian dilanjutkan dengan mengeksekusi script DML (*Data Manipulation Language*) berupa perintah INSERT INTO untuk memasukkan data dummy yang telah disiapkan.

6. Pembuatan Query SELECT, View, Trigger, Stored Procedure, dan Function.

Tahap akhir adalah membangun operasional dan programmabilitas database. Kami menyusun 10 query SELECT dengan kompleksitas bertingkat (sederhana, JOIN, subquery/CTE, dan agregasi), membuat 2 buah VIEW untuk keamanan ringkasan data, menerapkan STORED PROCEDURE untuk transaksi pembayaran, membuat TRIGGER untuk otomatisasi pemotongan stok produk, dan menyusun FUNCTION untuk klasifikasi ulasan pelanggan.

3.2. Entity Relationship Diagram (ERD)

Entity Relationship Diagram (ERD) berfungsi sebagai representasi visual yang memetakan rancangan logis dan keterkaitan data di dalam sistem. Model konseptual ini menerapkan standar notasi *Crow's Foot*, sebuah metode pemodelan yang memanfaatkan simbol kaki gagak sebagai penanda kapasitas kardinalitas relasi antar-objek. Di dalam basis data ini, terdapat 11 entitas inti yang saling terintegrasi, yakni: Pelanggan, Alamat, Produk, Kategori, Keranjang, Detail Keranjang, Pesanan, Detail Pesanan, Pembayaran, Pengiriman, dan *Review*. Jaringan relasi ini secara utuh merekam alur kerja transaksi pada ekosistem *e-commerce*, yang bergerak dari fase pemilihan komoditas oleh pembeli, penyimpanan sementara di keranjang, eksekusi *checkout*, hingga tahap penyelesaian pembayaran dan pengantaran logistik. Berikut adalah ilustrasi gambar ERD dengan notasi *Crow's Foot* pada platform *e-commerce* yang dibangun:



Gambar 3.1.1 Entity Relationship Diagram (ERD) Crow's Foot

3.3. Deskripsi Entitas dan Atribut

Berdasarkan skema model data konseptual yang telah digambarkan pada bagian sebelumnya, arsitektur basis data platform *e-commerce* ini dibangun oleh 12 entitas utama. Setiap entitas memiliki fungsi spesifik dalam menyimpan data operasional platform dan dilengkapi dengan beberapa atribut yang bertindak sebagai karakteristik data. Berikut adalah deskripsi fungsi dari masing-masing entitas beserta tabel spesifikasi atribut penyusunnya. &

1. Pelanggan:

Tabel Pelanggan berfungsi untuk menyimpan seluruh data identitas, kredensial, dan informasi personal dari setiap pengguna yang terdaftar di dalam platform, baik yang bertindak sebagai pembeli maupun penjual.

Nama Atribut	Jenis Key	Keterangan
id_pelanggan	Primary Key	ID unik untuk mengidentifikasi setiap pelanggan

nama_pelanggan	-	Nama asli pengguna sesuai kartu identitas.
email	-	Email pelanggan yang digunakan untuk hak akses masuk (<i>login</i>)
no_telepon	-	Nomor telepon pelanggan pengguna untuk keperluan verifikasi.
password	-	Kata sandi akun yang tersimpan dalam bentuk tersandi/terenkripsi.
status_akun	-	Menunjukkan status operasional akun pengguna (Aktif/Tidak Aktif)

2. Alamat:

Tabel Alamat berfungsi untuk menampung data lokasi pengantaran logistik paket pembeli. Entitas ini dipisahkan dari tabel pelanggan agar satu pengguna dapat menyimpan lebih dari satu lokasi pengiriman.

Nama Atribut	Jenis Key	Keterangan
id_alamat	Primary Key	ID unik untuk mengidentifikasi setiap baris data alamat
id_pelanggan	Foreign Key	Menghubungkan alamat ke entitas Pelanggan
nama_penerima	-	Nama orang yang akan menerima paket di lokasi tersebut.
no_hp	-	Nomor telepon orang yang menerima paket
alamat_lengkap	-	Detail nama jalan, nomor rumah, RT/RW, dan kelurahan/kecamatan
kode_pos	-	Kode pos wilayah alamat tujuan.
is_utama	-	Penanda alamat pengiriman utama pengguna (Ya/Tidak)

3. Wilayah:

Tabel Wilayah berfungsi untuk menampung data master lokasi administratif yang mencakup kode pos, kota, dan provinsi. Entitas ini dipisahkan dari tabel alamat agar memenuhi prinsip bentuk normal ketiga (3NF), sehingga mencegah terjadinya pengulangan data (redundansi) nama kota dan provinsi

untuk kode pos yang sama, serta menjaga konsistensi data wilayah di dalam sistem.

Nama Atribut	Jenis Key	Keterangan
kode_pos	Primary Key	Kode pos wilayah alamat tujuan. (unik)
provinsi	-	Nama provinsi tujuan pengiriman.
kota	-	Nama kota atau kabupaten tujuan pengiriman.

4. Kategori

Tabel Kategori berfungsi untuk mengelompokkan jenis komoditas produk guna memudahkan proses pencarian, penyaringan, dan klasifikasi barang di dalam aplikasi.

Nama Atribut	Jenis Key	Keterangan
id_kategori	Primary Key	ID unik yang mengelompokkan setiap kategori produk.
nama_kategori	-	Nama kategori produk
deskripsi_kategori	-	Penjelasan singkat mengenai cakupan produk di dalam kategori tersebut.

5. Produk

Tabel Produk untuk memuat seluruh informasi detail, spesifikasi, nilai jual, dan ketersediaan dari barang-barang yang dipasarkan di dalam platform.

Nama Atribut	Jenis Key	Keterangan
id_produk	Primary Key	ID unik untuk mengidentifikasi setiap item produk yang dijual.
id_kategori	Foreign Key	Menghubungkan produk ke entitas Kategori
nama_produk	-	Nama dari produk yang dijual.
deskripsi_produk	-	Informasi detail mengenai produk.
harga_produk	-	Nominal harga jual satuan dari produk terkait.
stok	-	Jumlah kuantitas fisik barang yang tersedia di gudang penyimpanan.

6. Keranjang

Tabel Keranjang berfungsi sebagai tempat penampungan data sementara (*temporary storage*) untuk melacak aktivitas belanja pembeli sebelum mereka memutuskan menuju fase pembuatan pesanan akhir.

Nama Atribut	Jenis Key	Keterangan
id_keranjang	Primary Key	ID unik untuk mengidentifikasi keranjang belanja milik pengguna.
id_pelanggan	Foreign Key	Menghubungkan keranjang ke pemiliknya di entitas Pelanggan
tanggal_dibuat	-	Waktu pertama kali keranjang belanja diinisialisasi oleh sistem.

7. Detail Keranjang

Tabel Detail Keranjang yang memecah hubungan banyak-ke-banyak (*Many-to-Many*) antara entitas Keranjang dan Produk, berfungsi mencatat daftar item barang beserta jumlahnya di dalam satu keranjang.

Nama Atribut	Jenis Key	Keterangan
id_detail_keranjang	Primary Key	ID unik untuk setiap baris item yang masuk ke dalam keranjang.
id_keranjang	Foreign Key	Menghubungkan Detail Keranjang ke entitas Kategori
id_produk	Foreign Key	Menghubungkan Detail Keranjang ke entitas Produk
jumlah	-	Jumlah unit produk yang ingin dimasukkan pembeli ke keranjang.

8. Pesanan

Tabel Pesanan berfungsi untuk mendokumentasikan rangkuman informasi transaksi utama, akumulasi biaya, dan status pelacakan ketika pembeli secara resmi melakukan konfirmasi belanja (*checkout*).

Nama Atribut	Jenis Key	Keterangan
id_pesanan	Primary Key	ID unik transaksi (nomor pesanan) yang diterbitkan otomatis saat <i>checkout</i> .
id_pelanggan	Foreign Key	Menghubungkan transaksi ke akun pembeli di entitas Pelanggan
id_alamat	Foreign Key	Menghubungkan transaksi ke lokasi

		tujuan pengiriman spesifik di entitas Alamat yang dipilih saat <i>checkout</i> .
tanggal_pesanan	-	Waktu pencatatan kapan transaksi tersebut dibuat oleh sistem.
total_harga	-	Akumulasi total biaya keseluruhan nilai barang yang harus dibayar.
status_pesanan	-	Kondisi transaksi saat ini (misalnya:Menunggu, Diproses, Selesai, Dibatalkan).

9. Detail Pesanan

Tabel Detail Pesanan berfungsi untuk menyimpan rincian item barang secara permanen (*snapshot data*) pada saat transaksi berhasil dikunci, sehingga riwayat harga lama tidak akan berubah meskipun harga di tabel produk diperbarui di kemudian hari.

Nama Atribut	Jenis Key	Keterangan
id_detail_pesanan	Primary Key	ID unik untuk setiap baris rincian item barang yang dibeli.
id_pesanan	Foreign Key	Menghubungkan Detail Pesanan ke entitas Pesanan
id_produk	Foreign Key	Menghubungkan Detail Pesanan ke entitas Produk
jumlah_beli	-	Jumlah unit barang yang resmi dibeli dalam pesanan tersebut.
harga_satuan	-	Catatan harga jual produk per unit tepat pada saat pembelian terjadi.
subtotal	-	Nilai total harga per baris item produk yang dibeli, dihitung secara otomatis dari hasil perkalian antara jumlah unit dengan harga per unit.

10. Pembayaran

Tabel Pembayaran berfungsi untuk mengelola dan memvalidasi rekaman aliran dana masuk atas pelunasan tagihan tagihan pesanan dari pembeli.

Nama Atribut	Jenis Key	Keterangan
--------------	-----------	------------

id_pembayaran	Primary Key	ID unik untuk mengidentifikasi bukti pembayaran
id_pesanan	Foreign Key	Menghubungkan pembayaran ke nota transaksi di entitas Pesanan
metode_pembayaran	-	Jenis pembayaran yang digunakan (misal: Transfer Bank, E-Wallet, COD)
tanggal_bayar	-	Waktu konfirmasi pembayaran diterima oleh sistem
jumlah_bayar	-	Nominal uang tunai yang dibayarkan oleh pelanggan
status_pembayaran	-	Kondisi status keuangan transaksi (misalnya: Lunas, Menunggu, Batalkan).

11. Pengiriman

Tabel Pengiriman berfungsi untuk mengelola data operasional kurir dan mencatat riwayat pelacakan (*tracking*) status distribusi fisik barang secara berkala, mulai dari penyerahan barang oleh penjual hingga paket tiba di alamat rumah pembeli.

Nama Atribut	Jenis Key	Keterangan
id_pengiriman	Primary Key	ID unik data pengiriman untuk keperluan pengarsipan logistik di sistem.
id_pesanan	Foreign Key	Menghubungkan data logistik ke dokumen transaksi di entitas Pesanan
kurir	-	Nama perusahaan penyedia jasa ekspedisi yang digunakan (misalnya: JNE, J&T, Sicepat).
nomor_resi	-	Kode nomor pelacakan resmi yang diterbitkan oleh pihak ekspedisi.

alamat_tujuan	-	Alamat lengkap pengiriman saat pesanan diproses
tanggal_pengiriman	-	Tanggal waktu paket diserahkan ke pihak ekspedisi
status_pengiriman	-	Status posisi paket (misal: Dikirim, Diterima, Belum Dikirim)

12. Review

Tabel Review berfungsi untuk menyimpan data penilaian (*review*) dari pembeli setelah transaksi dinyatakan selesai, guna mengumpulkan informasi tingkat kepuasan konsumen berupa skor angka (*rating*) serta ulasan tertulis (*testimoni*).

Nama Atribut	Jenis Key	Keterangan
id_review	Primary Key	ID unik catatan ulasan produk yang ditulis pengguna.
id_pelanggan	Foreign Key	Menghubungkan ulasan ke riwayat transaksi di entitas Pesanan.
id_produk	Foreign Key	Menghubungkan ulasan ke barang terkait di entitas Produk
rating	-	Nilai kepuasan berupa angka (skala bintang 1-5)
komentar	-	Isi ulasan atau teks testimoni tertulis dari pelanggan
tanggal_review	-	Tanggal waktu ulasan tersebut dikirimkan oleh pelanggan

3.4. Kardinalitas Relasi Antar Entitas

Kardinalitas menunjukkan jumlah hubungan antara satu entitas dengan entitas lainnya. Berikut adalah kardinalitas relasi pada sistem:

No	Entitas Asal	Entitas Tujuan	Kardinalitas	Penjelasan aturan bisnis (Relasi)
1	Pelanggan	Alamat	1:N (One to	Satu pelanggan dapat

			Many)	menyimpan lebih dari satu alamat pengiriman, sedangkan satu alamat hanya dimiliki oleh satu pelanggan.
2	Pelanggan	Keranjang	1:1 (One to One)	Setiap pelanggan memiliki satu keranjang aktif yang digunakan untuk menampung produk sebelum melakukan checkout.
3	Keranjang	Detail_Keranjang	1:N (One to Many)	Satu keranjang dapat berisi banyak item produk yang tersimpan pada tabel detail keranjang.
4	Produk	Detail_Keranjang	1:N (One to Many)	Satu produk dapat dimasukkan ke dalam banyak keranjang pelanggan yang berbeda.
5	Kategori	Produk	1:N (One to Many)	Satu kategori dapat memiliki banyak produk, sedangkan satu produk hanya berada dalam satu kategori.
6	Pelanggan	Pesanan	1:N (One to Many)	Satu pelanggan dapat membuat banyak pesanan selama menggunakan sistem.
7	Alamat	Pesanan	1:N (One to Many)	Satu alamat dapat digunakan sebagai alamat pengiriman pada beberapa pesanan yang dibuat pelanggan.
8	Pesanan	Detail_Pesanan	1:N (One to Many)	Satu pesanan dapat terdiri dari beberapa produk yang dicatat pada detail pesanan.
9	Produk	Detail_Pesanan	1:N (One to Many)	Satu produk dapat muncul pada banyak detail pesanan dari transaksi yang berbeda.
10	Pesanan	Pembayaran	1:1 (One to One)	Setiap pesanan memiliki satu data pembayaran yang

				mencatat metode, jumlah, dan status pembayaran.
11	Pesanan	Pengiriman	1:1 (One to One)	Setiap pesanan memiliki satu data pengiriman yang berisi informasi kurir, nomor resi, dan status pengiriman.
12	Wilayah	Alamat	1:N (One to Many)	Satu wilayah (kode pos) dapat digunakan oleh banyak alamat pelanggan, sedangkan satu alamat hanya berada pada satu wilayah.
13	Pelanggan	Review	1:N (One to Many)	Satu pelanggan dapat memberikan banyak ulasan terhadap produk yang pernah dibeli.
14	Produk	Review	1:N (One to Many)	Satu produk dapat menerima banyak ulasan dari pelanggan yang berbeda.

3.5. Normalisasi Data

Proses normalisasi dilakukan bertahap mulai dari Unnormalized Form (UNF) hingga Third Normal Form (3NF) untuk mengeliminasi redundansi data dan memastikan integritas data.

3.5.1. *Unnormalized Form (UNF)*

Pada bentuk UNF, seluruh data disimpan dalam satu tabel besar dengan atribut multi-nilai (nilai yang berulang dalam satu baris). Tabel UNF memiliki 50 kolom dan menggabungkan data pelanggan, produk, pesanan, pembayaran, pengiriman, dan review dalam satu baris. Berikut ini adalah masalah-masalah yang ada pada UNF:

- Terdapat kelompok berulang (repeating groups) yang berisi lebih dari satu nilai dalam satu sel.
- Redundansi data pelanggan: informasi nama, email diulang di setiap baris pesanan
- Redundansi data alamat: informasi alamat pengiriman berulang untuk setiap pesanan pelanggan yang sama.

- d. Redudansi data produk: informasi produk seperti nama produk, harga, stok, dan kategori berulang pada setiap detail pesanan.
- e. Beberapa atribut masih berisi lebih dari satu nilai dalam satu sel sehingga belum memenuhi syarat 1NF.

Sebagai contoh:

id_pesanan	id_detail_pesanan	id_produk	nama_produk	harga_produk
ORD-001	DTL-001, DTL-002	PRD-01, PRD-04	Kemeja Flannel Pria, Serum Retinol 20ml	150000, 85000

3.5.2. First Normal Form (1NF)

Untuk mencapai 1NF, setiap atribut multi-nilai dipecah sehingga setiap sel hanya berisi satu nilai (*atomic value*). Baris yang memiliki beberapa produk dipecah menjadi beberapa baris terpisah. Perubahan yang terjadi dari UNF ke 1NF adalah sebagai berikut:

- a. Seluruh atribut yang memiliki nilai lebih dari satu dalam satu sel (*multi-valued attribute*) dipisahkan sehingga setiap sel hanya berisi satu nilai (*atomic value*).
- b. Data yang sebelumnya tersimpan dalam bentuk kelompok berulang (*repeating groups*) dipecah menjadi beberapa baris terpisah.
- c. Atribut seperti **id_produk**, **nama_produk**, **jumlah**, **harga_satuan**, **subtotal**, dan atribut multi-nilai lainnya tidak lagi berisi daftar nilai dalam satu sel.
- d. Jumlah baris bertambah dari 20 baris (UNF) menjadi 24 baris (1NF) akibat proses pemisahan atribut multi-nilai.
- e. Setiap baris kini merepresentasikan satu data yang unik tanpa adanya kelompok berulang.

Masalah-masalah yang masih ada di 1NF:

- a. Redundansi data pelanggan: informasi pelanggan seperti nama, email, nomor telepon, dan status akun masih berulang pada setiap pesanan yang dilakukan pelanggan.
- b. Redundansi data alamat: informasi alamat pengiriman masih berulang pada setiap transaksi pelanggan yang sama.

- c. Redundansi data produk: informasi produk seperti nama produk, harga, stok, dan kategori masih berulang pada setiap detail pesanan.
- d. Data transaksi masih bercampur dengan data master sehingga berpotensi menimbulkan anomali saat proses insert, update, dan delete.

Sebagai contoh:

id_pesanan	id_detail_pesanan	id_produk	nama_produk	harga_produk
ORD-001	DTL-001	PRD-01	Kemeja Flannel Pria	150000
ORD-001	DTL-002	PRD-04	Serum Retinol 20ml	85000

3.5.3. Second Normal Form (2NF)

Untuk mencapai Second Normal Form (2NF), dilakukan pemisahan data berdasarkan entitas sehingga setiap atribut non-key bergantung sepenuhnya pada *primary key* tabelnya. Proses ini bertujuan untuk mengurangi redundansi data dan mencegah terjadinya anomali pada saat penyisipan, pembaruan, maupun penghapusan data. Perubahan yang terjadi dari First Normal Form (1NF) ke Second Normal Form (2NF) adalah sebagai berikut:

1. Data pelanggan dipisahkan ke dalam Tabel Pelanggan yang terdiri atas atribut *id_pelanggan*, *nama_pelanggan*, *email*, *no_telepon*, *password*, dan *status_akun*.
2. Data alamat dipisahkan ke dalam Tabel Alamat yang terhubung dengan Tabel Pelanggan melalui atribut *id_pelanggan* sebagai *foreign key*.
3. Data kategori produk dipisahkan ke dalam Tabel Kategori untuk menghindari pengulangan informasi kategori pada setiap produk.
4. Data produk dipisahkan ke dalam Tabel Produk yang terdiri atas atribut *id_produk*, *id_kategori*, *nama_produk*, *deskripsi*, *harga*, *stok*, *gambar*, dan *status*.
5. Data transaksi dipisahkan ke dalam Tabel Pesanan sebagai tabel utama yang menyimpan informasi pemesanan pelanggan.

6. Rincian produk yang dipesan disimpan dalam Tabel Detail Pesanan yang memiliki hubungan dengan Tabel Pesanan dan Tabel Produk melalui *foreign key*.
7. Data keranjang belanja dipisahkan ke dalam Tabel Keranjang dan Tabel Detail Keranjang untuk menyimpan informasi produk yang dipilih pelanggan sebelum melakukan transaksi.
8. Data pembayaran dipisahkan ke dalam Tabel Pembayaran untuk menyimpan informasi terkait proses pembayaran pesanan.
9. Data pengiriman dipisahkan ke dalam Tabel Pengiriman untuk menyimpan informasi pengiriman pesanan kepada pelanggan.
10. Data ulasan pelanggan dipisahkan ke dalam Tabel Review untuk menyimpan informasi penilaian dan komentar pelanggan terhadap produk yang telah dibeli.

Hasil normalisasi pada tahap 2NF menghasilkan beberapa tabel yang terpisah berdasarkan entitas masing-masing sehingga redundansi data dapat diminimalkan dan struktur basis data menjadi lebih terorganisasi. Contoh hasil 2NF sebagai berikut:

1. Tabel pesanan

id_pesanan
ORD-001

2. Tabel detail pesanan

id_detail_pesanan	id_pesanan	id_produk
DTL-001	ORD-001	PRD-01
DTL-002	ORD-001	PRD-04

3. Tabel produk

id_produk	nama_produk	harga produk
PRD-01	Kemeja Flannel Pria	150000
PRD-04	Serum Retinol 20ml	85000

3.5.4. Third Normal Form (3NF)

Untuk mencapai 3NF, semua transitive dependency dihilangkan. Atribut non-key yang bergantung pada atribut non-key lainnya dipisahkan. Perubahan yang terjadi dari 2NF ke 3NF adalah sebagai berikut:

- Ditemukan transitive dependency pada tabel Alamat: kode_pos -> kota, provinsi
- Tabel Wilayah dibuat sebagai tabel baru dengan Primary Key kode_pos

kode_pos	kota	provinsi
40111	Bandung	Jawa Barat
60111	Surabaya	Jawa Timur
10110	Jakarta Pusat	DKI Jakarta
53122	Purwokerto	Jawa Tengah
12190	Jakarta Selatan	DKI Jakarta
55281	Yogyakarta	DI Yogyakarta
55222	Yogyakarta	DI Yogyakarta
55283	Sleman	DI Yogyakarta
20123	Medan	Sumatera Utara
65111	Malang	Jawa Timur
16122	Bogor	Jawa Barat
50132	Semarang	Jawa Tengah
17111	Bekasi	Jawa Barat
40222	Bandung	Jawa Barat
16411	Depok	Jawa Barat
12980	Jakarta Selatan	DKI Jakarta

- Pada tabel Alamat, kolom kota dan provinsi dihapus digantikan dengan FK ke Wilayah (kode_pos)

id alamat	id pelanggan	nama penerima	no hp	kode_pos	alamat lengkap	is utama
ALM-01	PLG-01	Siti Aminah	8123456789	40111	Jl. Anggrek No. 12, Kel. Merdeka	Ya
ALM-02	PLG-02	Budi Santoso	8139876543	60111	Perum Gading Indah Blok C/4	Ya
ALM-03	PLG-03	Rumah Ibu Dewi	8571122334	10110	Gang Kelinci No. 45, RT 02/RW 05	Tidak
ALM-05	PLG-04	Ahmad Fauzi	8215566778	53122	Komplek dosen Unsoed No. B9	Ya
ALM-06	PLG-05	Kantor Rizky	8998877665	12190	Sudirman Central Business District Tbk	Tidak
ALM-07	PLG-06	Fitriani	8112233445	55281	Kost Putri Muslimah, Jl. Gading No.3	Ya
ALM-08	PLG-07	Hendra Wijaya	8523344556	55222	Jl. Pemuda No. 88, Klitren	Ya
ALM-09	PLG-08	Toko Buku Andi	8784455667	55283	Jl. Gejayan No. 10B, Condongcatur	Tidak
ALM-10	PLG-09	Mega Utami	8129988776	20123	Perumahan Bumi Asri Blok F/12	Ya
ALM-11	PLG-10	Rian Hidayat	8137766554	65111	Jl. Merdeka No. 17, Kel. Kidul	Ya
ALM-12	PLG-11	Anita Sari	8564433221	16122	Kost Orange, Gg. Sayur No. 2	Ya
ALM-13	PLG-12	Eko Prasetyo	8126655443	50132	Jl. Pemuda Selatan No. 14	Ya
ALM-14	PLG-13	Sri Wahyuni	8138899001	17111	Perum Hijau Daun Blok A1 No. 5	Ya
ALM-15	PLG-14	Bambang T.	8575544332	40222	Jl. Gatot Subroto No. 99	Ya
ALM-16	PLG-15	Indah Permata	8221122334	16411	Town House cluster Diamond No. 2	Ya
ALM-04	PLG-03	Kantor Dewi	8218888999	12980	Menara Imperium Lt. 12, Kuningan	Ya

- Dengan pemisahan ini, perubahan nama kota/provinsi hanya perlu dilakukan di satu tempat.

Hasil akhir 3NF menghasilkan 12 tabel yang telah teroptimasi tanpa redundansi dan anomali data

3.6. Kamus Data (Data Dictionary)

Berikut adalah data dictionary lengkap untuk setiap tabel dalam sistem basis data e-commerce:

1. Tabel Pelanggan

Nama Kolom	Tipe Data	Constraint	Deskripsi
id_pelanggan	VARCHAR (10)	PRIMARY KEY	Identifikasi unik pelanggan (format: PLG-XX)
nama_pelanggan	VARCHAR (100)	NOT NULL	Nama lengkap pelanggan
email	VARCHAR (150)	UNIQUE, NOT NULL	Alamat email pelanggan (unik)
no_telepon	VARCHAR (20)	NOT NULL	Nomor telepon/HP pelanggan
password	VARCHAR (255)	NOT NULL	Password terenkripsi (bcrypt/pbkdf2)
status_akun	ENUM	NOT NULL, DEFAULT 'Aktif'	Status: Aktif / Tidak Aktif

2. Tabel Wilayah

Nama Kolom	Tipe Data	Constraint	Deskripsi
kode_pos	VARCHAR (10)	PRIMARY KEY	Identifikasi unik untuk kode pos wilayah
kota	VARCHAR (100)	NOT NULL	Nama kota atau kabupaten tempat tinggal
provinsi	VARCHAR (100)	NOT NULL	Nama provinsi wilayah terkait

3. Tabel Alamat

Nama Kolom	Tipe Data	Constraint	Deskripsi
id_alamat	VARCHAR (20)	PRIMARY KEY	Identifikasi unik entitas data alamat
id_pelanggan	VARCHAR	FOREIGN	Merujuk ke

	(20)	KEY, NOT NULL	id_pelanggan pada tabel pelanggan
nama_penerima	VARCHAR (100)	NOT NULL	Nama orang penerima paket pengiriman
no_hp	VARCHAR (20)	NOT NULL	Nomor telepon penerima paket
alamat_lengkap	TEXT	NOT NULL	Detail jalan, nomor rumah, RT/RW, dan kelurahan
kode_pos	VARCHAR (10)	FOREIGN KEY, NOT NULL	Merujuk ke kode_pos pada tabel wilayah
is_utama	ENUM('Ya', 'Tidak')	NOT NULL, DEFAULT 'Tidak'	Menandai apakah menjadi alamat pengiriman utama

4. Tabel Kategori

Nama Kolom	Tipe Data	Constraint	Deskripsi
id_kategori	VARCHAR (10)	PRIMARY KEY	Identifikasi unik entitas data alamat
nama_kategori	VARCHAR (100)	NOT NULL	Merujuk ke id_pelanggan pada tabel pelanggan
deskripsi_kategori	TEXT	NOT NULL	Nama orang penerima paket pengiriman

5. Tabel Produk

Nama Kolom	Tipe Data	Constraint	Deskripsi
id_produk	VARCHAR (10)	PRIMARY KEY	Identifikasi unik entitas data alamat
id_kategori	VARCHAR (10)	PRIMARY KEY	Identifikasi unik entitas data alamat
nama_produk	VARCHAR (100)	NOT NULL	Merujuk ke id_pelanggan pada tabel pelanggan
deskripsi_kategori	TEXT	NOT NULL	Detail deskripsi spesifikasi barang produk

harga_produk	INT	NOT NULL	Harga jual produk dalam satuan Rupiah
stok	INT	NOT NULL	Jumlah persediaan barang yang tersedia di gudang
gambar_produk	VARCHAR(255)	NOT NULL	Jalur direktori URL file gambar aset produk
status_produk	ENUM('Tersedia', 'Habis')	NOT NULL, DEFAULT 'Tersedia'	NOT NULL, DEFAULT 'Tersedia'

6. Tabel Keranjang

Nama Kolom	Tipe Data	Constraint	Deskripsi
id_keranjang	VARCHAR(10)	PRIMARY KEY	Identifikasi unik keranjang belanja (format: CRJ-XX)
id_pelanggan	VARCHAR(10)	FOREIGN KEY, NOT NULL	Merujuk ke id_pelanggan pada tabel pelanggan
tanggal_dibuat	DATE	NOT NULL, DEFAULT(CURRENT DATE)	Tanggal sesi keranjang belanja pertama kali dibuat

7. Tabel Detail Keranjang

Nama Kolom	Tipe Data	Constraint	Deskripsi
id_detail_keranjang	VARCHAR(10)	PRIMARY KEY	Identifikasi unik detail item keranjang (format: DCRJ-XX)
id_keranjang	VARCHAR(10)	FOREIGN KEY, NOT NULL	Merujuk ke id_keranjang pada tabel keranjang
id_produk	VARCHAR(10)	FOREIGN KEY, NOT NULL	Merujuk ke id_produk pada tabel produk
jumlah	INT	NOT NULL	Kuantitas unit produk yang dicadangkan pembeli

8. Tabel Pesanan

Nama Kolom	Tipe Data	Constraint	Deskripsi
id_pesanan	VARCHAR (10)	PRIMARY KEY	Identifikasi unik nota transaksi (format: ORD-XXX)
id_pelanggan	VARCHAR (10)	FOREIGN KEY, NOT NULL	Merujuk ke id_pelanggan pada tabel pelanggan
id_alamat	VARCHAR (10)	FOREIGN KEY, NOT NULL	Merujuk ke id_alamat pada tabel alamat
tanggal_pesanan	INT	NOT NULL	Tanggal dikeluarkannya invoice transaksi belanja
total_harga	INT	NOT NULL	Akumulasi total nominal biaya belanja keseluruhan
status_pesanan	VARCHAR (30)	NOT NULL	Status alur pesanan (Menunggu/Diproses/Dikirim/Selesai/Batal)

9. Tabel Detail Pesanan

Nama Kolom	Tipe Data	Constraint	Deskripsi
id_detail_pesanan	VARCHAR (10)	PRIMARY KEY	Identifikasi unik item rincian pesanan (format: DT-XXX)
id_pesanan	VARCHAR (10)	FOREIGN KEY, NOT NULL	Merujuk ke id_pesanan pada tabel pesanan
id_produk	VARCHAR (10)	FOREIGN KEY, NOT NULL	Merujuk ke id_produk pada tabel produk
jumlah_beli	INT	NOT NULL	Volume atau kuantitas produk yang dibeli konsumen
harga_satuan	INT	NOT NULL	Harga dasar produk per unit saat transaksi dikunci
sub_total	INT	NOT NULL	Nilai hasil kalkulasi dari (jumlah_beli × harga_satuan)

10. Tabel Pembayaran

Nama Kolom	Tipe Data	Constraint	Deskripsi
id_pembayaran	VARCHAR(10)	PRIMARY KEY	Identifikasi unik kwitansi pembayaran (format: PMB-XX)
id_pesanan	VARCHAR(10)	FOREIGN KEY, NOT NULL	Merujuk ke id_pesanan pada tabel pesanan
metode_pembayaran	VARCHAR(50)	NOT NULL	Media transaksi instan (Transfer Bank/E-Wallet/Kartu Kredit)
tanggal_bayar	DATE	NULL	Tanggal pembeli melakukan konfirmasi kiriman dana
jumlah_bayar	INT	NOT NULL	Jumlah nominal uang tunai yang ditransfer ke kasir
status_pembayaran	VARCHAR(30)	NOT NULL	Menyimpan status validasi aliran dana dari setiap transaksi

11. Tabel Pengiriman

Nama Kolom	Tipe Data	Constraint	Deskripsi
id_pengiriman	VARCHAR(10)	PRIMARY KEY	Identifikasi unik manifes logistik paket (format: JKT-XX)
id_pesanan	VARCHAR(10)	FOREIGN KEY, NOT NULL	Merujuk ke id_pesanan pada tabel pesanan
kurir	VARCHAR(50)	NOT NULL	Nama perusahaan jasa logistik (J&T / JNE / SiCepat / Ninja)
nomor_resi	VARCHAR(100)	NULL	Kode tracking

			pengiriman resmi dari pihak ekspedisi
alamat_tujuan	TEXT	NOT NULL	Salinan riil alamat bongkar muat dropping paket
tanggal_pengiriman	DATE	NULL	Tanggal penyerahan barang toko ke tangan ekspedisi
status_pengiriman	VARCHAR(30)	NOT NULL	Posisi fisik barang kurir (Belum Dikirim / Dikirim / Diterima)

12. Tabel Review

Nama Kolom	Tipe Data	Constraint	Deskripsi
id_review	VARCHAR(10)	PRIMARY KEY	Identifikasi unik ulasan kepuasan (format: RVW-XX)
id_pelanggan	VARCHAR(10)	FOREIGN KEY, NOT NULL	Merujuk ke id_pelanggan pada tabel pelanggan
id_produk	VARCHAR(10)	FOREIGN KEY, NOT NULL	Merujuk ke id_produk pada tabel produk
rating	INT	CHECK (BETWEEN 1 AND 5), NOT NULL	Skor penilaian tingkat kepuasan fungsional produk
komentar	TEXT	NULL	Isi kalimat ulasan testimoni tertulis dari konsumen
tanggal_review	DATE	NOT NULL, DEFAULT(CURRENT_DATE)	Tanggal submit data ulasan produk ke dalam sistem

BAB IV

IMPLEMENTASI

4.1. Implementasi Database

Basis data yang dibuat diberi nama marketplace1 dan terdiri atas dua belas (12) tabel utama yang saling berelasi, yaitu: wilayah, pelanggan, alamat, kategori, produk, keranjang, detail_keranjang, pesanan, detail_pesanan, pembayaran, pengiriman, dan review. Rancangan terbaru ini telah mengalami normalisasi ke bentuk Normal Ketiga (3NF) dengan memecah kolom kota dan provinsi dari tabel alamat menjadi tabel tersendiri bernama wilayah, sehingga tidak terjadi anomali data akibat ketergantungan transitif (transitive dependency) antara kode_pos dengan kota/provinsi. Selain itu, seluruh primary key dan foreign key pada rancangan terbaru menggunakan tipe data VARCHAR(10) dengan format kode (misalnya PLG-01, PRD-01, ORD-001) menggantikan tipe INT AUTO_INCREMENT pada rancangan sebelumnya, sehingga ID lebih mudah dibaca dan ditelusuri secara manual.

Implementasi DDL (Data Definition Language)

Data Definition Language (DDL) merupakan kumpulan pernyataan SQL yang digunakan untuk mendefinisikan dan mengelola struktur basis data, seperti pembuatan basis data, tabel, constraint, dan indeks. Perintah DDL utama yang digunakan dalam implementasi ini adalah CREATE DATABASE, CREATE TABLE, dan CREATE INDEX.

4.1.1. Pembuatan Database

Langkah pertama implementasi adalah pembuatan basis data dengan perintah berikut:

```
CREATE DATABASE marketplace;  
USE marketplace;
```

Perintah CREATE DATABASE digunakan untuk membuat basis data baru bernama marketplace, sedangkan USE marketplace digunakan untuk menjadikan basis data tersebut sebagai konteks aktif sehingga seluruh perintah DDL berikutnya akan diterapkan pada basis data marketplace1.

4.1.2. Pembuatan Tabel

Tabel-tabel dalam basis data marketplace dibuat menggunakan pernyataan CREATE TABLE. Urutan pembuatan tabel disusun berdasarkan ketergantungan foreign key, dimulai dari tabel wilayah yang menjadi tabel hasil normalisasi 3NF dan menjadi referensi bagi tabel alamat. Berikut adalah detail pembuatan setiap tabel beserta atribut yang dimilikinya.

a) Tabel Wilayah

Tabel wilayah merupakan tabel baru hasil pemecahan (dekomposisi) dari tabel alamat untuk memenuhi bentuk normal ketiga (3NF). Kolom kota dan provinsi sebelumnya bergantung secara transitif terhadap kode_pos di dalam tabel alamat, sehingga dipisahkan menjadi tabel tersendiri agar setiap kode_pos hanya disimpan satu kali dan tidak terjadi duplikasi data kota/provinsi.

```
CREATE TABLE wilayah (  
    kode_pos VARCHAR(10) PRIMARY KEY,  
    kota VARCHAR(100) NOT NULL,  
    provinsi VARCHAR(100) NOT NULL  
);
```

Field / Kolom	Tipe Data	Constraint / Keterangan
kode_pos	VARCHAR(10)	PRIMARY KEY kode_pos sebagai identifikasi unik wilayah
kota	VARCHAR(100)	NOT NULL
provinsi	VARCHAR(100)	NOT NULL

b) Tabel Pelanggan

Tabel pelanggan menyimpan data identitas pengguna yang terdaftar pada sistem marketplace. Primary key id_pelanggan kini menggunakan tipe VARCHAR(10) dengan format kode seperti PLG-01, dan kolom status_akun memiliki nilai default 'aktif' sehingga tidak perlu diisi secara eksplisit saat pendaftaran akun baru.

```
CREATE TABLE pelanggan (
    id_pelanggan VARCHAR(10) PRIMARY KEY,
    nama_pelanggan VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    no_telepon VARCHAR(20),
    password VARCHAR(255) NOT NULL,
    status_akun VARCHAR(20) DEFAULT 'aktif'
);
```

Field / Kolom	Tipe Data	Constraint / Keterangan
id_pelanggan	VARCHAR(10)	PRIMARY KEY format kode, contoh: PLG-01
nama_pelanggan	VARCHAR(100)	NOT NULL – nama tidak boleh kosong
email	VARCHAR(100)	UNIQUE NOT NULL email unik per pelanggan
no_telepon	VARCHAR(20)	Opsional
password	VARCHAR(255)	NOT NULL disimpan dalam bentuk hash
status_akun	VARCHAR(20)	DEFAULT 'aktif'

c) Tabel Alamat

Tabel alamat menyimpan satu atau lebih alamat pengiriman yang dimiliki oleh setiap pelanggan. Setelah normalisasi 3NF, kolom kota dan provinsi tidak lagi disimpan langsung pada tabel ini, melainkan diakses melalui relasi foreign key kode_pos menuju tabel wilayah.

```
CREATE TABLE alamat (
    id_alamat VARCHAR(10) PRIMARY KEY,
    id_pelanggan VARCHAR(10) NOT NULL,
    nama_penerima VARCHAR(100) NOT NULL,
    no_hp VARCHAR(20),
    alamat_lengkap TEXT,
    kode_pos VARCHAR(10) NOT NULL,
```

```

        is_utama          BOOLEAN DEFAULT FALSE,
        FOREIGN KEY (id_pelanggan) REFERENCES
        pelanggan(id_pelanggan),
        FOREIGN KEY (kode_pos) REFERENCES wilayah(kode_pos)
    );

```

Field / Kolom	Tipe Data	Constraint / Keterangan
id_alamat	VARCHAR(10)	PRIMARY KEY format kode, contoh: ALM-01
id_pelanggan	VARCHAR(10)	FOREIGN KEY pelanggan(id_pelanggan)
nama_penerima	VARCHAR(100)	NOT NULL
no_hp	VARCHAR(20)	Opsional
alamat_lengkap	TEXT	Detail alamat lengkap
kode_pos	VARCHAR(10)	NOT NULL; FOREIGN KEY wilayah(kode_pos)
is_utama	BOOLEAN	DEFAULT FALSE flag alamat utama

d) Tabel Kategori

```

CREATE TABLE kategori (
    id_kategori          VARCHAR(10) PRIMARY KEY,
    nama_kategori        VARCHAR(100) NOT NULL,
    deskripsi_kategori   TEXT
);

```

Field / Kolom	Tipe Data	Constraint / Keterangan
id_kategori	VARCHAR(10)	PRIMARY KEY format kode, contoh: KAT-01
nama_kategori	VARCHAR(100)	NOT NULL
deskripsi_kategori	TEXT	Opsional

e) Tabel Produk

```
CREATE TABLE produk (
    id_produk          VARCHAR(10) PRIMARY KEY,
    id_kategori        VARCHAR(10) NOT NULL,
    nama_produk        VARCHAR(150) NOT NULL,
    deskripsi_produk   TEXT,
    harga_produk       INT NOT NULL,
    stok               INT NOT NULL,
    gambar_produk      VARCHAR(255),
    status_produk      VARCHAR(20),
    FOREIGN KEY (id_kategori) REFERENCES kategori(id_kategori)
);
```

Field / Kolom	Tipe Data	Constraint / Keterangan
id_produk	VARCHAR(10)	PRIMARY KEY format kode, contoh: PRD-01
id_kategori	VARCHAR(10)	FOREIGN KEY kategori(id_kategori)
nama_produk	VARCHAR(150)	NOT NULL
deskripsi_produk	TEXT	Opsional
harga_produk	INT	NOT NULL3 nilai harga dalam rupiah (tanpa desimal)
stok	INT	NOT NULL
gambar_produk	VARCHAR(255)	Path atau URL gambar
status_produk	VARCHAR(20)	Tersedia / habis / nonaktif

f) Tabel Keranjang

```
CREATE TABLE keranjang (
    id_keranjang       VARCHAR(10) PRIMARY KEY,
    id_pelanggan       VARCHAR(10) NOT NULL,
    tanggal_dibuat     DATE DEFAULT (CURRENT_DATE),
    FOREIGN KEY (id_pelanggan) REFERENCES pelanggan(id_pelanggan)
);
```

Field / Kolom	Tipe Data	Constraint / Keterangan
id_keranjang	VARCHAR(10)	PRIMARY KEY – format kode, contoh: KRJ-01
id_pelanggan	VARCHAR(10)	NOT NULL; FOREIGN KEY pelanggan(id_pelanggan)
tanggal_dibuat	DATE	DEFAULT (CURRENT_DATE) otomatis diisi tanggal hari ini

g) Tabel Detail_keranjang

```
CREATE TABLE detail_keranjang (
    id_detail_keranjang VARCHAR(10) PRIMARY KEY,
    id_keranjang        VARCHAR(10) NOT NULL,
    id_produk           VARCHAR(10) NOT NULL,
    jumlah              INT NOT NULL,
    FOREIGN KEY (id_keranjang) REFERENCES
    keranjang(id_keranjang),
    FOREIGN KEY (id_produk) REFERENCES produk(id_produk)
);
```

Field / Kolom	Tipe Data	Constraint / Keterangan
id_detail_keranjang	VARCHAR(10)	PRIMARY KEY format kode, contoh: DKJ-01
id_keranjang	VARCHAR(10)	NOT NULL; FOREIGN KEY keranjang(id_keranjang)
id_produk	VARCHAR(10)	NOT NULL; FOREIGN KEY produk(id_produk)
jumlah	INT	NOT NULL jumlah item yang ditambahkan ke keranjang

h) Tabel Pesanan

```
CREATE TABLE pesanan (
    id_pesanan          VARCHAR(10) PRIMARY KEY,
    id_pelanggan        VARCHAR(10) NOT NULL,
    id_alamat           VARCHAR(10) NOT NULL,
    tanggal_pesanan     DATE DEFAULT (CURRENT_DATE),
```

```

total_harga      INT NOT NULL,
status_pesanan   VARCHAR(30),
FOREIGN KEY (id_pelanggan) REFERENCES
pelanggan(id_pelanggan),
FOREIGN KEY (id_alamat) REFERENCES alamat(id_alamat)
);

```

Field / Kolom	Tipe Data	Constraint / Keterangan
id_pesanan	VARCHAR(10)	PRIMARY KEY format kode, contoh: ORD-001
id_pelanggan	VARCHAR(10)	NOT NULL; FOREIGN KEY pelanggan(id_pelanggan)
id_alamat	VARCHAR(10)	NOT NULL; FOREIGN KEY alamat(id_alamat)
tanggal_pesanan	DATE	DEFAULT (CURRENT_DATE)
total_harga	INT	NOT NULL – total nilai pesanan dalam rupiah
status_pesanan	VARCHAR(30)	Selesai / Diproses / Menunggu / Dibatalkan

i) Tabel Detail_pesanan

```

CREATE TABLE detail_pesanan (
    id_detail_pesanan VARCHAR(10) PRIMARY KEY,
    id_pesanan         VARCHAR(10) NOT NULL,
    id_produk          VARCHAR(10) NOT NULL,
    jumlah_beli        INT NOT NULL,
    harga_satuan       INT NOT NULL,
    subtotal           INT NOT NULL,
    FOREIGN KEY (id_pesanan) REFERENCES pesanan(id_pesanan),
    FOREIGN KEY (id_produk) REFERENCES produk(id_produk)
);

```

Field / Kolom	Tipe Data	Constraint / Keterangan
id_detail_pesanan	VARCHAR(10)	PRIMARY KEY format kode, contoh: DPS-01

id_pesanan	VARCHAR(10)	NOT NULL; FOREIGN KEY pesanan(id_pesanan)
id_produk	VARCHAR(10)	NOT NULL; FOREIGN KEY produk(id_produk)
jumlah_beli	INT	NOT NULL kuantitas produk yang dibeli
harga_satuan	INT	NOT NULL harga per unit saat transaksi
subtotal	INT	NOT NULL hasil jumlah_beli × harga_satuan

j) Tabel Pembayaran

```
CREATE TABLE pembayaran (
    id_pembayaran    VARCHAR(10) PRIMARY KEY,
    id_pesanan       VARCHAR(10) NOT NULL,
    metode_pembayaran VARCHAR(50),
    tanggal_bayar    DATE,
    jumlah_bayar     INT,
    status_pembayaran VARCHAR(30),
    FOREIGN KEY (id_pesanan) REFERENCES pesanan(id_pesanan)
);
```

k) Tabel Pengiriman

```
CREATE TABLE pengiriman (
    id_pengiriman    VARCHAR(10) PRIMARY KEY,
    id_pesanan       VARCHAR(10) NOT NULL,
    kurir            VARCHAR(50),
    nomor_resi       VARCHAR(100),
    alamat_tujuan    TEXT,
    tanggal_pengiriman DATE,
    status_pengiriman VARCHAR(30),
    FOREIGN KEY (id_pesanan) REFERENCES pesanan(id_pesanan)
);
```

l) Tabel Review

```
CREATE TABLE review (
    id_review        VARCHAR(10) PRIMARY KEY,
    id_pelanggan     VARCHAR(10) NOT NULL,
    id_produk        VARCHAR(10) NOT NULL,
```



```

rating          INT CHECK (rating BETWEEN 1 AND 5),
komentar        TEXT,
tanggal_review  DATETIME DEFAULT (CURRENT_DATE),
FOREIGN KEY (id_pelanggan) REFERENCES
pelanggan(id_pelanggan),
FOREIGN KEY (id_produk) REFERENCES produk(id_produk)
);

```

4.1.3. Implementasi Constraint

Constraint merupakan aturan yang diterapkan pada kolom atau tabel untuk menjaga integritas dan konsistensi data. Tabel berikut merangkum seluruh constraint yang diterapkan dalam basis data marketplace.

Tabel	Jenis Constraint	Kolom	Keterangan
wilayah	PRIMARY KEY	kode_pos	Identifikasi unik wilayah
wilayah	NOT NULL	kota, provinsi	Wajib diisi
pelanggan	PRIMARY KEY	id_pelanggan	Identifikasi unik pelanggan
pelanggan	UNIQUE + NOT NULL	email	Email tidak boleh duplikat
pelanggan	NOT NULL	nama_pelanggan, password	Wajib diisi
pelanggan	DEFAULT	status_akun	Nilai default 'aktif'
alamat	FOREIGN KEY (2x)	id_pelanggan, kode_pos	Referensi ke pelanggan dan wilayah
alamat	DEFAULT	is_utama	Nilai default FALSE
produk	FOREIGN KEY	id_kategori	Referensi ke tabel kategori
produk	NOT NULL	harga_produk, stok	Wajib diisi, nilai tidak boleh kosong
review	CHECK	rating	Nilai 1 s.d. 5 saja
keranjang	DEFAULT	tanggal_dibuat	DEFAULT (CURRENT_DATE)
pesanan	FOREIGN KEY (2x)	id_pelanggan, id_alamat	Referensi ke pelanggan dan alamat

detail_pesanan	FOREIGN KEY (2x)	id_pesanan, id_produk	Referensi ke pesanan dan produk
pembayaran	FOREIGN KEY	id_pesanan	Referensi ke tabel pesanan
pengiriman	FOREIGN KEY	id_pesanan	Referensi ke tabel pesanan

Secara keseluruhan, basis data marketplace1 mengimplementasikan empat belas (14) FOREIGN KEY yang membentuk jaringan relasi antar tabel, bertambah satu relasi dibanding rancangan sebelumnya karena adanya tabel wilayah, serta satu (1) CHECK constraint pada kolom rating di tabel review. Seluruh primary key kini menggunakan tipe VARCHAR(10) dengan format kode yang ditentukan secara manual, menggantikan AUTO_INCREMENT pada rancangan sebelumnya.

4.1.4. Implementasi Index

Indeks merupakan struktur data tambahan yang dibuat untuk mempercepat proses pencarian (query) pada tabel berukuran besar. MySQL secara otomatis membuat indeks pada setiap kolom PRIMARY KEY dan UNIQUE. Selain indeks otomatis tersebut, dapat ditambahkan indeks manual pada kolom-kolom yang sering digunakan sebagai filter atau kondisi JOIN.

```
-- Indeks pada kolom yang sering difilter
CREATE INDEX idx_produk_kategori ON produk (id_kategori);
CREATE INDEX idx_pesanan_pelanggan ON pesanan (id_pelanggan);
CREATE INDEX idx_pesanan_status ON pesanan (status_pesanan);
CREATE INDEX idx_review_produk ON review (id_produk);
CREATE INDEX idx_alamat_pelanggan ON alamat (id_pelanggan);
CREATE INDEX idx_alamat_kode_pos ON alamat (kode_pos);
CREATE INDEX idx_keranjang_pelanggan ON keranjang (id_pelanggan);
```

Nama Indeks	Tabel	Manfaat
idx_produk_kategori	produk	Mempercepat filter produk berdasarkan kategori
idx_pesanan_pelanggan	pesanan	Mempercepat pencarian riwayat pesanan per pelanggan

idx_pesanan_status	pesanan	Mempercepat filter pesanan berdasarkan status
idx_review_produk	review	Mempercepat pengambilan review per produk
idx_alamat_pelanggan	alamat	Mempercepat pencarian alamat pengiriman pelanggan
idx_alamat_kode_pos	alamat	Mempercepat JOIN antara alamat dan wilayah
idx_keranjang_pelanggan	keranjang	Mempercepat akses keranjang belanja aktif

4.1.5. Insert Data Dummy

Data dummy dimasukkan ke dalam basis data untuk mensimulasikan kondisi nyata sistem marketplace dan memverifikasi bahwa relasi antar tabel berfungsi dengan benar. Data dummy disusun mengikuti urutan penambahan yang mempertimbangkan ketergantungan antar tabel (dependency order), dimulai dari tabel yang tidak memiliki foreign key hingga tabel yang bergantung pada tabel lainnya.

Urutan pengisian data dummy adalah sebagai berikut: (1) wilayah, (2) kategori, (3) pelanggan, (4) produk, (5) alamat, (6) keranjang, (7) detail_keranjang, (8) pesanan, (9) detail_pesanan, (10) pembayaran, (11) pengiriman, dan (12) review.

4.1.6. Data Dummy Tabel Pelanggan

Data dummy tabel pelanggan berisi lima belas akun pengguna dengan seluruh status akun aktif, mencakup pelanggan dari berbagai kota di Indonesia untuk mensimulasikan kondisi nyata sistem marketplace.

```
INSERT INTO pelanggan (id_pelanggan, nama_pelanggan, email,
                        no_telepon, password, status_akun)
VALUES
  ('PLG-01', 'Siti Aminah', 'sitiaminah@gmail.com', '08123456789',
    '$2b$12$K3j8...', 'Aktif'),
  ('PLG-02', 'Budi Santoso', 'budis@yahoo.com', '08139876543', 'budi_santoso2026!', 'Aktif'),
  ('PLG-03', 'Dewi Lestari', 'dewi.les@gmail.com',
```

```

        '08571122334', 'pbkdf2_sha256$...dewilestari99', 'Aktif'),
('PLG-04','Ahmad Fauzi', 'fauzi_ahmad@outlook.com',
        '08215566778', '$2b$12$7eR9...', 'Aktif'),
('PLG-05','Rizky Hidayat', 'rizky_hid@gmail.com',
        '08998877665', 'RizkyHidayat_Secure99!', 'Aktif'),
('PLG-06','Fitriani', 'fitri.ani@gmail.com',
        '08112233445', 'fitrianicantik123', 'Aktif'),
('PLG-07','Hendra Wijaya', 'hendraw@gmail.com',
        '08523344556', '$2b$12$Z1x2...', 'Aktif'),
('PLG-08','Andi Wijaya', 'andiw99@gmail.com', '08784455667',
        'andi_wijaya_rahasia', 'Aktif'),
('PLG-09','Mega Utami', 'mega_utami@gmail.com', '08129988776',
        'pbkdf2_sha256$...megautami_09', 'Aktif'),
('PLG-10','Rian Hidayat', 'rian_hid@gmail.com',
        '08137766554', '$2b$12$Q8w7...', 'Aktif'),
('PLG-11','Anita Sari', 'anita.sari@gmail.com', '08564433221',
        'anitasari_purwokerto', 'Aktif'),
('PLG-12','Eko Prasetyo', 'ekopras@gmail.com',
        '08126655443', 'pbkdf2_sha256$...ekoprasetyo99', 'Aktif'),
('PLG-13','Sri Wahyuni', 'sri.wahyuni@gmail.com', '08138899001',
        'sriwahyuni_hijaul', 'Aktif'),
('PLG-14','Bambang T.', 'bambangt@gmail.com', '08575544332',
        'Bambang_GatSu99', 'Aktif'),
('PLG-15','Indah Permata', 'indah.per@gmail.com',
        '08221122334', '$2b$12$A1s2...', 'Aktif')

```

ID	Nama	Email	No. Telepon	Status
PLG-01	Siti Aminah	sitiaminah@gmail.com	08123456789	Aktif
PLG-02	Budi Santoso	budis@yahoo.com	08139876543	Aktif
PLG-03	Dewi Lestari	dewi.les@gmail.com	08571122334	Aktif
PLG-04	Ahmad Fauzi	fauzi_ahmad@outlook.com	08215566778	Aktif
PLG-05	Rizky Hidayat	rizky_hid@gmail.com	08998877665	Aktif

PLG-06	Fitriani	fitri.ani@gmail.com	0811223344 5	Aktif
PLG-07	Hendra Wijaya	hendraw@gmail.com	0852334455 6	Aktif
PLG-08	Andi Wijaya	andiw99@gmail.com	0878445566 7	Aktif
PLG-09	Mega Utami	mega_utami@gmail. com	0812998877 6	Aktif
PLG-10	Rian Hidayat	rian_hid@gmail.com	0813776655 4	Aktif
PLG-11	Anita Sari	anita.sari@gmail.co m	0856443322 1	Aktif
PLG-12	Eko Prasetyo	ekopras@gmail.com	0812665544 3	Aktif
PLG-13	Sri Wahyuni	sri.wahyuni@gmail.c om	0813889900 1	Aktif
PLG-14	Bambang T.	bambangt@gmail.co m	0857554433 2	Aktif
PLG-15	Indah Permata	indah.per@gmail.co m	0822112233 4	Aktif

4.1.7. Data Dummy Tabel Produk

Sebelum memasukkan data produk, diperlukan data kategori sebagai referensi foreign key. Basis data marketplace memiliki empat (4) kategori produk sebagai berikut:

```
INSERT INTO kategori (id_kategori, nama_kategori, deskripsi_kategori)
VALUES
('KAT-01', 'Fashion', 'Pakaian pria & wanita'),
```

```
( 'KAT-02', 'Kuliner',      'Makanan & minuman'),
( 'KAT-03', 'Kecantikan', 'Perawatan wajah & tubuh'),
( 'KAT-04', 'Rumah Tangga', 'Perlengkapan rumah');
```

Selanjutnya data dummy untuk tabel produk (5 produk dari 4 kategori berbeda):

```
INSERT INTO produk (id_produk, id_kategori, nama_produk, deskripsi_produk,
                    harga_produk, stok, gambar_produk, status_produk)
VALUES
( 'PRD-01', 'KAT-01', 'Kemeja Flannel Pria',  'Kemeja katun kotak-kotak',
  150000, 45, 'kemeja.jpg',      'Tersedia'),
( 'PRD-02', 'KAT-02', 'Sambal Cumi Asin 200g', 'Sambal rumahan premium',
  45000, 80, 'sambalcumi.jpg',  'Tersedia'),
( 'PRD-03', 'KAT-03', 'Lip Tint Cherry',      'Pewarna bibir natural',
  90000, 200, 'liptint.jpg',     'Tersedia'),
( 'PRD-04', 'KAT-03', 'Serum Retinol 20ml',   'Serum anti-aging',
  85000, 120, 'retinol.jpg',     'Tersedia'),
( 'PRD-05', 'KAT-04', 'Sapu Lantai Rayon',     'Sapu lantai anti rontok',
  40000, 50, 'sapurayon.jpg',    'Tersedia');
```

Data dummy pesanan terdiri dari dua puluh (20) transaksi dengan berbagai kombinasi status (Selesai, Diproses, Menunggu, Dibatalkan) yang mencerminkan alur bisnis nyata sistem marketplace. Karena tabel alamat kini berelasi dengan tabel wilayah hasil normalisasi 3NF, data wilayah harus disisipkan terlebih dahulu sebelum data alamat:

```
INSERT INTO wilayah (kode_pos, kota, provinsi)
VALUES
( '40111', 'Bandung',      'Jawa Barat'),
( '60111', 'Surabaya',     'Jawa Timur'),
( '10110', 'Jakarta Pusat', 'DKI Jakarta'),
( '12980', 'Jakarta Selatan', 'DKI Jakarta'),
( '53122', 'Purwokerto',   'Jawa Tengah'),
( '12190', 'Jakarta Selatan', 'DKI Jakarta'),
( '55281', 'Yogyakarta',   'DI Yogyakarta'),
( '55222', 'Yogyakarta',   'DI Yogyakarta'),
( '55283', 'Sleman',       'DI Yogyakarta'),
( '20123', 'Medan',        'Sumatera Utara'),
( '65111', 'Malang',       'Jawa Timur'),
( '16122', 'Bogor',        'Jawa Barat'),
( '50132', 'Semarang',     'Jawa Tengah'),
( '17111', 'Bekasi',       'Jawa Barat'),
( '40222', 'Bandung',      'Jawa Barat'),
( '16411', 'Depok',        'Jawa Barat');
```

Selanjutnya disisipkan data alamat pengiriman (16 alamat dari 15 pelanggan), yang masing-masing merujuk pada kode_pos di tabel wilayah:

```
INSERT INTO alamat (id_alamat, id_pelanggan, nama_penerima, no_hp,
                    alamat_lengkap, kode_pos, is_utama)
VALUES
('ALM-01','PLG-01','Siti Aminah', '08123456789', 'Jl. Anggrek No. 12,
            Kel. Merdeka', '40111', TRUE),
('ALM-02','PLG-02','Budi Santoso', '08139876543', 'Perum Gading Indah
            Blok C/4', '60111', TRUE),
('ALM-03','PLG-03','Rumah Ibu Dewi','08571122334', 'Gang Kelinci No. 45,
            RT 02/RW 05', '10110', FALSE),
('ALM-04','PLG-03','Kantor Dewi', '08218888999', 'Menara Imperium Lt.
            12, Kuningan', '12980', TRUE),
('ALM-05','PLG-04','Ahmad Fauzi', '08215566778', 'Komplek Dosen Unsoed
            No. B9', '53122', TRUE),
('ALM-06','PLG-05','Kantor Rizky', '08998877665', 'Sudirman Central
            Business District Tbk', '12190', FALSE),
('ALM-07','PLG-06','Fitriani', '08112233445', 'Kost Putri Muslimah,
            Jl. Gading No.3', '55281', TRUE),
('ALM-08','PLG-07','Hendra Wijaya', '08523344556', 'Jl. Pemuda No. 88,
            Klitren', '55222', TRUE),
('ALM-09','PLG-08','Toko Buku Andi','08784455667', 'Jl. Gejayan No. 10B,
            Condongcatur', '55283', FALSE),
('ALM-10','PLG-09','Mega Utami', '08129988776', 'Perumahan Bumi Asri
            Blok F/12', '20123', TRUE),
('ALM-11','PLG-10','Rian Hidayat', '08137766554', 'Jl. Merdeka No. 17,
            Kel. Kidul', '65111', TRUE),
('ALM-12','PLG-11','Anita Sari', '08564433221', 'Kost Orange, Gg.
            Sayur No. 2', '16122', TRUE),
('ALM-13','PLG-12','Eko Prasetyo', '08126655443', 'Jl. Pemuda Selatan
            No. 14', '50132', TRUE),
('ALM-14','PLG-13','Sri Wahyuni', '08138899001', 'Perum Hijau Daun Blok
            A1 No. 5', '17111', TRUE),
('ALM-15','PLG-14','Bambang T.', '08575544332', 'Jl. Gatot Subroto No.
            99', '40222', TRUE),
('ALM-16','PLG-15','Indah Permata', '08221122334', 'Town House Cluster
            Diamond No. 2', '16411', TRUE);
```

Selanjutnya data dummy dua puluh (20) pesanan. Kolom total_harga, harga_satuan, dan subtotal kini bertipe INT (sebelumnya decimal), sehingga nilai uang ditulis tanpa angka desimal:

```
INSERT INTO pesanan (id_pesanan, id_pelanggan, id_alamat, tanggal_pesanan,
                    total_harga, status_pesanan)
VALUES
('ORD-001','PLG-01','ALM-01','2026-06-11', 235000, 'Selesai'),
('ORD-002','PLG-02','ALM-02','2026-06-11', 90000, 'Selesai'),
```

```
( 'ORD-003', 'PLG-03', 'ALM-03', '2026-06-12', 240000, 'Diproses'),
( 'ORD-004', 'PLG-04', 'ALM-05', '2026-06-12', 120000, 'Menunggu'),
( 'ORD-005', 'PLG-01', 'ALM-01', '2026-06-12', 45000, 'Selesai'),
( 'ORD-006', 'PLG-05', 'ALM-06', '2026-06-13', 135000, 'Selesai'),
( 'ORD-007', 'PLG-06', 'ALM-07', '2026-06-13', 150000, 'Dibatalkan'),
( 'ORD-008', 'PLG-07', 'ALM-08', '2026-06-13', 170000, 'Selesai'),
( 'ORD-009', 'PLG-08', 'ALM-09', '2026-06-13', 80000, 'Selesai'),
( 'ORD-010', 'PLG-09', 'ALM-10', '2026-06-13', 180000, 'Diproses'),
( 'ORD-011', 'PLG-10', 'ALM-11', '2026-06-14', 340000, 'Selesai'),
( 'ORD-012', 'PLG-11', 'ALM-12', '2026-06-14', 45000, 'Selesai'),
( 'ORD-013', 'PLG-12', 'ALM-13', '2026-06-15', 150000, 'Diproses'),
( 'ORD-014', 'PLG-13', 'ALM-14', '2026-06-15', 170000, 'Menunggu'),
( 'ORD-015', 'PLG-14', 'ALM-15', '2026-06-15', 40000, 'Selesai'),
( 'ORD-016', 'PLG-15', 'ALM-16', '2026-06-15', 175000, 'Selesai'),
( 'ORD-017', 'PLG-02', 'ALM-02', '2026-06-16', 150000, 'Diproses'),
( 'ORD-018', 'PLG-03', 'ALM-04', '2026-06-16', 45000, 'Selesai'),
( 'ORD-019', 'PLG-04', 'ALM-05', '2026-06-16', 135000, 'Diproses'),
( 'ORD-020', 'PLG-10', 'ALM-11', '2026-06-16', 80000, 'Selesai');
```

ID Pesanan	Pelanggan	Tgl Pesanan	Total (Rp)	Status	Produk Utama
ORD-001	Siti Aminah	2026-06-11	235.000	Selesai	PRD-01 + PRD-04
ORD-002	Budi Santoso	2026-06-11	90.000	Selesai	PRD-02 (2x)
ORD-003	Dewi Lestari	2026-06-12	240.000	Diproses	PRD-03 + PRD-01
ORD-004	Ahmad Fauzi	2026-06-12	120.000	Menunggu	PRD-05 (3x)
ORD-005	Siti Aminah	2026-06-12	45.000	Selesai	PRD-02
ORD-006	Rizky Hidayat	2026-06-13	135.000	Selesai	PRD-02 (3x)
ORD-007	Fitriani	2026-06-13	150.000	Dibatalkan	PRD-01
ORD-008	Hendra Wijaya	2026-06-13	170.000	Selesai	PRD-04 (2x)

ORD-009	Andi Wijaya	2026-06-13	80.000	Selesai	PRD-05 (2x)
ORD-010	Mega Utami	2026-06-13	180.000	Diproses	PRD-03 (2x)
ORD-011-020	(berbagai pelanggan)	2026-06-14/16	40.000–340.000	Selesai / Diproses / Menunggu	Peragam produk

Total data dummy yang berhasil disisipkan pada tahap DML: 16 wilayah, 15 pelanggan, 16 alamat, 4 kategori, 5 produk, 20 pesanan, 24 detail pesanan, 15 keranjang, dan 24 detail keranjang. Seluruh data dapat diakses dan diverifikasi melalui perintah SELECT setelah proses insert selesai.

4.1.8. Data Dummy Tabel Detail Pesanan

Dalam rangka mensimulasikan alur transaksi penjualan dan menyediakan bahan analisis visual, kami mengintegrasikan sekumpulan data tiruan (*dummy data*) ke dalam tabel detail_pesanan. Data ini mencakup variasi kuantitas pembelian serta keberagaman produk yang dipesan oleh pelanggan dalam beberapa sesi transaksi yang berbeda. Struktur data ini nantinya akan digunakan sebagai fondasi utama dalam menarik kesimpulan terkait produk terlaris (*best-selling*) dan total pendapatan. Adapun rincian data transaksi tersebut dipaparkan pada tabel di bawah ini:

```
INSERT INTO detail_pesanan (id_detail_pesanan, id_pesanan, id_produk,
jumlah_beli, harga_satuan, subtotal) VALUES
('DTL-001', 'ORD-001', 'PRD-01', 1, 150000, 150000),
('DTL-002', 'ORD-001', 'PRD-04', 1, 85000, 85000),
('DTL-003', 'ORD-002', 'PRD-02', 2, 45000, 90000),
('DTL-004', 'ORD-003', 'PRD-03', 1, 90000, 90000),
('DTL-005', 'ORD-003', 'PRD-01', 1, 150000, 150000),
('DTL-006', 'ORD-004', 'PRD-05', 3, 40000, 120000),
('DTL-007', 'ORD-005', 'PRD-02', 1, 45000, 45000),
('DTL-008', 'ORD-006', 'PRD-02', 3, 45000, 135000),
('DTL-009', 'ORD-007', 'PRD-01', 1, 150000, 150000),
('DTL-010', 'ORD-008', 'PRD-04', 2, 85000, 170000),
('DTL-011', 'ORD-009', 'PRD-05', 2, 40000, 80000),
('DTL-012', 'ORD-010', 'PRD-03', 2, 90000, 180000),
```

```
( 'DTL-013', 'ORD-011', 'PRD-01', 1, 150000, 150000),
( 'DTL-014', 'ORD-011', 'PRD-03', 2, 90000, 180000),
( 'DTL-015', 'ORD-012', 'PRD-02', 1, 45000, 45000),
( 'DTL-016', 'ORD-013', 'PRD-01', 1, 150000, 150000),
( 'DTL-017', 'ORD-014', 'PRD-04', 2, 85000, 170000),
( 'DTL-018', 'ORD-015', 'PRD-05', 1, 40000, 40000),
( 'DTL-019', 'ORD-016', 'PRD-03', 1, 90000, 90000),
( 'DTL-020', 'ORD-016', 'PRD-04', 1, 85000, 85000),
( 'DTL-021', 'ORD-017', 'PRD-01', 1, 150000, 150000),
( 'DTL-022', 'ORD-018', 'PRD-02', 1, 45000, 45000),
( 'DTL-023', 'ORD-019', 'PRD-02', 3, 45000, 135000),
( 'DTL-024', 'ORD-020', 'PRD-05', 2, 40000, 80000);
SELECT * FROM DETAIL_PESANAN;
```

ID Detail Pesanan	ID Pesanan	ID Produk	Jumlah Beli	Harga Satuan	Subtotal
DTL-001	ORD-001	PRD-01	1	150.000	150.000
DTL-002	ORD-001	PRD-04	1	85.000	85.000
DTL-003	ORD-002	PRD-02	2	45.000	90.000
DTL-004	ORD-003	PRD-03	1	90.000	90.000
DTL-005	ORD-003	PRD-01	1	150.000	150.000
DTL-006	ORD-004	PRD-05	3	40.000	120.000
DTL-007	ORD-005	PRD-02	1	45.000	45.000
DTL-008	ORD-006	PRD-02	3	45.000	135.000
DTL-009	ORD-007	PRD-01	1	150.000	150.000

DTL-010	ORD-008	PRD-04	2	85.000	170.000
DTL-011	ORD-009	PRD-05	2	40.000	80.000
DTL-012	ORD-010	PRD-03	2	90.000	180.000
DTL-013	ORD-011	PRD-01	1	150.000	150.000
DTL-014	ORD-011	PRD-03	2	90.000	180.000
DTL-015	ORD-012	PRD-02	1	45.000	45.000
DTL-016	ORD-013	PRD-01	1	150.000	150.000

4.1.9. Data Dummy Tabel Keranjang

Sesi simulasi pengisian data (*data insertion*) dilanjutkan pada entitas penampung sementara sebelum terjadinya transaksi, yaitu tabel keranjang (*shopping cart*). Pengisian data *dummy* pada tabel ini bertujuan untuk memvalidasi fungsi fitur pra-pembelian, di mana sistem harus mampu mencatat aktivitas pelanggan saat memilih produk sebelum mereka melakukan proses *checkout*. Data yang dimasukkan mengaitkan secara langsung antara identitas unik pelanggan (*id_pelanggan*) dengan waktu pembuatan keranjang (*tanggal_dibuat*). Proses pengujian ini krusial untuk memastikan bahwa sistem dapat mempertahankan riwayat belanja pengguna berdasarkan lini masa tertentu. Struktur dan detail data hasil eksekusi perintah SQL INSERT tersebut dapat dilihat pada visualisasi tabel di bawah ini:

```
INSERT INTO keranjang (id_keranjang, id_pelanggan, tanggal_dibuat) VALUES
('CRJ-01', 'PLG-01', '2026-06-01'),
('CRJ-02', 'PLG-02', '2026-06-05'),
('CRJ-03', 'PLG-03', '2026-06-09'),
('CRJ-04', 'PLG-04', '2026-06-11'),
('CRJ-05', 'PLG-05', '2026-06-10'),
('CRJ-06', 'PLG-06', '2026-06-12'),
('CRJ-07', 'PLG-07', '2026-06-11'),
('CRJ-08', 'PLG-08', '2026-06-12'),
('CRJ-09', 'PLG-09', '2026-06-11'),
('CRJ-10', 'PLG-10', '2026-06-12'),
('CRJ-11', 'PLG-11', '2026-06-13'),
('CRJ-12', 'PLG-12', '2026-06-14'),
```

```

('CRJ-13', 'PLG-13', '2026-06-12'),
('CRJ-14', 'PLG-14', '2026-06-14'),
('CRJ-15', 'PLG-15', '2026-06-13');
SELECT * FROM KERANJANG;

```

ID Keranjang	ID Pelanggan	Tanggal Dibuat
CRJ-01	PLG-01	01/06/2026
CRJ-02	PLG-02	05/06/2026
CRJ-03	PLG-03	09/06/2026
CRJ-04	PLG-04	11/06/2026
CRJ-05	PLG-05	10/06/2026

4.1.10. Data Dummy Tabel Detail Keranjang

Untuk mensimulasikan dinamika pemilihan produk oleh calon pembeli, dilakukan pengisian data *dummy* ke dalam tabel detail_keranjang. Fokus utama dari pengisian data di tahap ini adalah memvalidasi pencatatan kolom jumlah, yang merepresentasikan kuantitas dari setiap item produk yang diminati pelanggan dalam satu sesi belanja. Data simulasi ini mencerminkan kondisi riil di mana pelanggan bisa menambahkan, menumpuk, atau memisahkan item produk di dalam keranjang mereka sebelum melakukan akumulasi harga. Kumpulan data tiruan yang telah berhasil disimpan dan dipanggil kembali dari basis data melalui perintah SQL ini dapat dicermati pada tabel di bawah ini:

```

INSERT INTO detail_keranjang (id_detail_keranjang, id_keranjang, id_produk,
jumlah) VALUES
('DCRJ-01', 'CRJ-01', 'PRD-01', 1),
('DCRJ-02', 'CRJ-01', 'PRD-04', 1),
('DCRJ-03', 'CRJ-02', 'PRD-02', 2),
('DCRJ-04', 'CRJ-03', 'PRD-03', 1),
('DCRJ-05', 'CRJ-03', 'PRD-01', 1),

```

```

('DCRJ-06', 'CRJ-04', 'PRD-05', 3),
('DCRJ-07', 'CRJ-01', 'PRD-02', 1),
('DCRJ-08', 'CRJ-05', 'PRD-02', 3),
('DCRJ-09', 'CRJ-06', 'PRD-01', 1),
('DCRJ-10', 'CRJ-07', 'PRD-04', 2),
('DCRJ-11', 'CRJ-08', 'PRD-05', 2),
('DCRJ-12', 'CRJ-09', 'PRD-03', 2),
('DCRJ-13', 'CRJ-10', 'PRD-01', 1),
('DCRJ-14', 'CRJ-10', 'PRD-03', 2),
('DCRJ-15', 'CRJ-11', 'PRD-02', 1),
('DCRJ-16', 'CRJ-12', 'PRD-01', 1),
('DCRJ-17', 'CRJ-13', 'PRD-04', 2),
('DCRJ-18', 'CRJ-14', 'PRD-05', 1),
('DCRJ-19', 'CRJ-15', 'PRD-03', 1),
('DCRJ-20', 'CRJ-15', 'PRD-04', 1),
('DCRJ-21', 'CRJ-02', 'PRD-01', 1),
('DCRJ-22', 'CRJ-03', 'PRD-02', 1),
('DCRJ-23', 'CRJ-04', 'PRD-02', 3),
('DCRJ-24', 'CRJ-10', 'PRD-05', 2);
SELECT * FROM DETAIL_KERANJANG;

```

ID Detail Keranjang	ID Keranjang	ID Produk	Jumlah
DCRJ-01	CRJ-01	PRD-01	1
DCRJ-02	CRJ-01	PRD-04	1
DCRJ-03	CRJ-02	PRD-02	2
DCRJ-04	CRJ-03	PRD-03	1
DCRJ-05	CRJ-03	PRD-01	1
DCRJ-06	CRJ-04	PRD-05	3
DCRJ-07	CRJ-01	PRD-02	1

DCRJ-08	CRJ-05	PRD-02	3
DCRJ-09	CRJ-06	PRD-01	1
DCRJ-10	CRJ-07	PRD-04	2
DCRJ-11	CRJ-08	PRD-05	2

4.1.11. Data Dummy Tabel Pembayaran

Tahap akhir dari siklus transaksi dalam sistem e-commerce disimulasikan melalui pengisian data tiruan (*dummy data*) ke dalam tabel pembayaran. Penginputan data pada sub-bab ini bertujuan untuk memverifikasi kemampuan sistem dalam mencatat alur pelunasan pesanan yang dilakukan oleh pelanggan. Data yang dimasukkan mencakup keberagaman mekanisme pembayaran (*metode_pembayaran*), pencatatan waktu finansial (*tanggal_bayar*), serta nominal yang disetorkan (*jumlah_bayar*). Selain itu, pengujian ini sangat krusial untuk menguji logika kondisional sistem terhadap status transaksi; di mana pesanan yang telah dibayar akan berstatus 'Lunas', sedangkan transaksi yang belum diselesaikan akan mencatat nilai NULL pada tanggal bayar, angka 0 pada jumlah bayar, dan berstatus 'Menunggu'. Struktur data hasil eksekusi perintah SQL INSERT dan pemanggilan SELECT pada tabel ini dijabarkan sebagai berikut:

```
INSERT INTO pembayaran (id_pembayaran, id_pesanan, metode_pembayaran,
tanggal_bayar, jumlah_bayar, status_pembayaran) VALUES
('PMB-01', 'ORD-001', 'Transfer Bank', '2026-06-10', 235000, 'Lunas'),
('PMB-02', 'ORD-002', 'E-Wallet (OVO)', '2026-06-10', 90000, 'Lunas'),
('PMB-03', 'ORD-003', 'Transfer Bank', '2026-06-11', 240000, 'Lunas'),
('PMB-04', 'ORD-004', 'Transfer Bank', NULL, 0, 'Menunggu'),
('PMB-05', 'ORD-005', 'E-Wallet (Gopay)', '2026-06-11', 45000, 'Lunas'),
('PMB-06', 'ORD-006', 'Kartu Kredit', '2026-06-12', 135000, 'Lunas'),
('PMB-07', 'ORD-008', 'Transfer Bank', '2026-06-12', 170000, 'Lunas'),
('PMB-08', 'ORD-009', 'E-Wallet (Gopay)', '2026-06-12', 80000, 'Lunas'),
('PMB-09', 'ORD-010', 'E-Wallet (Dana)', '2026-06-13', 180000, 'Lunas'),
('PMB-10', 'ORD-011', 'Transfer Bank', '2026-06-13', 340000, 'Lunas'),
('PMB-11', 'ORD-012', 'E-Wallet (OVO)', '2026-06-13', 45000, 'Lunas'),
('PMB-12', 'ORD-013', 'Transfer Bank', '2026-06-14', 150000, 'Lunas'),
('PMB-13', 'ORD-014', 'Transfer Bank', NULL, 0, 'Menunggu'),
('PMB-14', 'ORD-015', 'E-Wallet (Gopay)', '2026-06-14', 40000, 'Lunas'),
```

```
( 'PMB-15', 'ORD-016', 'Kartu Kredit', '2026-06-14', 175000, 'Lunas'),
( 'PMB-16', 'ORD-017', 'Transfer Bank', '2026-06-15', 150000, 'Lunas'),
( 'PMB-17', 'ORD-018', 'E-Wallet (Dana)', '2026-06-15', 45000, 'Lunas'),
( 'PMB-18', 'ORD-019', 'Transfer Bank', '2026-06-15', 135000, 'Lunas'),
( 'PMB-19', 'ORD-020', 'E-Wallet (Gopay)', '2026-06-15', 80000, 'Lunas');
SELECT * FROM PEMBAYARAN;
```

ID Pembayaran	ID Pesanan	Metode Pembayaran	Tanggal Bayar	Jumlah Bayar	Status Pembayaran
PMB-01	ORD-001	Transfer Bank	10/06/2026	235.000	Lunas
PMB-02	ORD-002	E-Wallet (OVO)	10/06/2026	90.000	Lunas
PMB-03	ORD-003	Transfer Bank	11/06/2026	240.000	Lunas
PMB-04	ORD-004	Transfer Bank	NULL	0	Menunggu
PMB-05	ORD-005	E-Wallet (Gopay)	11/06/2026	45.000	Lunas
PMB-06	ORD-006	Kartu Kredit	12/06/2026	135.000	Lunas
PMB-07	ORD-008	Transfer Bank	12/06/2026	170.000	Lunas
PMB-08	ORD-009	E-Wallet (Gopay)	12/06/2026	80.000	Lunas
PMB-09	ORD-010	E-Wallet (Dana)	13/06/2026	180.000	Lunas

PMB-10	ORD-011	Transfer Bank	13/06/2026	340.000	Lunas
PMB-11	ORD-012	E-Wallet (OVO)	13/06/2026	45.000	Lunas
PMB-12	ORD-013	Transfer Bank	14/06/2026	150.000	Lunas
PMB-13	ORD-014	Transfer Bank	NULL	0	Menunggu

4.1.12. Data Dummy Tabel Pengiriman

Untuk mensimulasikan alur distribusi barang setelah proses pemesanan dan pembayaran berhasil divalidasi oleh sistem, dilakukan pengisian data tiruan (*dummy data*) ke dalam tabel pengiriman. Penginputan data pada sub-bab ini bertujuan untuk menguji fungsionalitas fitur logistik, pelacakan paket, serta ketepatan integrasi data kurir. Data *dummy* ini dirancang dengan menyertakan atribut penting seperti nama penyedia jasa logistik (kurir), nomor pelacakan unik (nomor_resi), lokasi tujuan (alamat_tujuan), hingga lini masa keberangkatan (tanggal_pengiriman). Selain itu, pengujian ini krusial untuk memastikan sistem dapat memperbarui dan membedakan siklus operasional pengiriman secara berkala melalui kolom status_pengiriman (seperti status 'Dikirim' untuk paket yang masih dalam perjalanan dan 'Diterima' untuk paket yang sudah sampai ke tangan konsumen). Struktur data hasil eksekusi perintah SQL INSERT dan pemanggilan SELECT pada tabel ini dipaparkan sebagai berikut:

```
INSERT INTO pengiriman (id_pengiriman, id_pesanan, kurir, nomor_resi,
alamat_tujuan, tanggal_pengiriman, status_pengiriman) VALUES
('JKT-01', 'ORD-001', 'J&T Express', 'JNT100239481', 'Jl. Anggrek No. 12,
Bandung', '2026-06-10', 'Diterima'),
('JKT-02', 'ORD-002', 'SiCepat Anteraja', 'SCP992834111', 'Perum Gading
Indah Blok C/4, Surabaya', '2026-06-11', 'Diterima'),
('JKT-03', 'ORD-003', 'JNE Express', 'JNE003928114', 'Gang Kelinci No. 45,
Jakarta Pusat', '2026-06-11', 'Dikirim'),
('JKT-04', 'ORD-005', 'J&T Express', 'JNT100239499', 'Jl. Anggrek No. 12,
Bandung', '2026-06-11', 'Diterima'),
('JKT-05', 'ORD-006', 'SiCepat Anteraja', 'SCP992834150', 'SCBD, Jakarta
Selatan', '2026-06-12', 'Diterima'),
('JKT-06', 'ORD-008', 'JNE Express', 'JNE003928155', 'Jl. Pemuda No. 88,
Yogyakarta', '2026-06-12', 'Diterima'),
```



```

('JKT-07', 'ORD-009', 'J&T Express', 'JNT100239512', 'Jl. Gejayan No. 10B, Sleman', '2026-06-12', 'Diterima'),
('JKT-08', 'ORD-010', 'SiCepat Anteraja', 'SCP992834199', 'Perum Bumi Asri Blok F/12, Medan', '2026-06-13', 'Dikirim'),
('JKT-09', 'ORD-011', 'J&T Express', 'JNT100239555', 'Jl. Merdeka No. 17, Malang', '2026-06-13', 'Diterima'),
('JKT-10', 'ORD-012', 'Ninja Xpress', 'NJA882911002', 'Kost Orange, Gg. Sayur No. 2, Bogor', '2026-06-13', 'Diterima'),
('JKT-11', 'ORD-013', 'JNE Express', 'JNE003928210', 'Jl. Pemuda Selatan No. 14, Semarang', '2026-06-14', 'Dikirim'),
('JKT-12', 'ORD-015', 'J&T Express', 'JNT100239600', 'Jl. Gatot Subroto No. 99, Bandung', '2026-06-14', 'Diterima'),
('JKT-13', 'ORD-016', 'SiCepat Anteraja', 'SCP992834211', 'Cluster Diamond No. 2, Depok', '2026-06-14', 'Diterima'),
('JKT-14', 'ORD-017', 'JNE Express', 'JNE003928315', 'Perum Gading Indah Blok C/4, Surabaya', '2026-06-15', 'Dikirim'),
('JKT-15', 'ORD-018', 'SiCepat Anteraja', 'SCP992834300', 'Menara Imperium Lt. 12, Jakarta', '2026-06-15', 'Diterima'),
('JKT-16', 'ORD-019', 'J&T Express', 'JNT100239611', 'Komplek dosen Unsoed, Purwokerto', '2026-06-15', 'Dikirim'),
('JKT-17', 'ORD-020', 'J&T Express', 'JNT100239699', 'Jl. Merdeka No. 17, Malang', '2026-06-15', 'Diterima');
SELECT * FROM PENGIRIMAN;

```

ID Pengirim	ID Pesanan	Kurir	Nomor Resi	Alamat Tujuan	Tanggal Pengiriman	Status Pengiriman
JKT-01	ORD-001	J&T Express	JNT100239481	Jl. Anggrek No. 12, Bandung	10/06/2026	Diterima
JKT-02	ORD-002	SiCepat Anteraja	SCP992834111	Perum Gading Indah Blok C/4, Surabaya	11/06/2026	Diterima

JKT-03	ORD-003	JNE Expres s	JNE003928114	Gang Kelinci No. 45, Jakarta Pusat	11/06/2026	Dikirim
JKT-04	ORD-005	J&T Expres s	JNT100239499	Jl. Anggrek No. 12, Bandung	11/06/2026	Diterima
JKT-05	ORD-006	SiCepat Anteraja	SCP992834150	SCBD, Jakarta Selatan	12/06/2026	Diterima
JKT-06	ORD-008	JNE Expres s	JNE003928155	Jl. Pemuda No. 88, Yogyakarta	12/06/2026	Diterima

4.1.13. Data Dummy Tabel Review

Tahap pengujian basis data pasca-transaksi disimulasikan melalui pengisian data tiruan (*dummy data*) ke dalam tabel review. Penginputan data pada sub-bab ini bertujuan untuk memverifikasi fungsionalitas fitur umpan balik (*user feedback*) dan sistem penilaian (*rating*) yang diberikan oleh pelanggan terhadap produk yang telah mereka beli. Data *dummy* ini dirancang dengan mengombinasikan data kuantitatif berupa skala penilaian angka (*rating*) serta data kualitatif berupa ulasan tekstual konkrit (komentar). Melalui skenario ini, sistem diuji kemampuannya dalam merekam persepsi kepuasan konsumen secara *real-time* berdasarkan lini masa tertentu (*tanggal_review*). Pengujian ini sangat

penting untuk memastikan bahwa sistem nantinya dapat mengalkulasi rata-rata penilaian produk guna membentuk reputasi toko yang kredibel. Struktur data hasil eksekusi perintah SQL INSERT dan pemanggilan SELECT pada tabel ini dijabarkan sebagai berikut:

```
INSERT INTO review (id_review, id_pelanggan, id_produk, rating, komentar,
tanggal_review) VALUES
('RVW-01', 'PLG-01', 'PRD-01', 5, 'Kainnya adem sekali', '2026-06-11'),
('RVW-02', 'PLG-01', 'PRD-04', 5, 'serumnya cocok', '2026-06-11'),
('RVW-03', 'PLG-02', 'PRD-02', 4, 'Sambalnya mantap, cumi tidak alot',
'2026-06-10'),
('RVW-04', 'PLG-01', 'PRD-02', 5, 'Langganan, selalu enak makan pake ini',
'2026-06-11'),
('RVW-05', 'PLG-05', 'PRD-02', 3, 'Rasa oke tapi pengiriman agak lama',
'2026-06-12'),
('RVW-06', 'PLG-07', 'PRD-04', 4, 'Istri saya suka, kerutan berkurang
katanya', '2026-06-12'),
('RVW-07', 'PLG-08', 'PRD-05', 5, 'Sapunya kokoh, gagang mantap',
'2026-06-13'),
('RVW-08', 'PLG-10', 'PRD-01', 4, 'Kemejanya pas', '2026-06-14'),
('RVW-09', 'PLG-10', 'PRD-03', 4, 'liptint buat kado adek', '2026-06-14'),
('RVW-10', 'PLG-11', 'PRD-02', 5, 'Enak banget, pedesnya nampol',
'2026-06-14'),
('RVW-11', 'PLG-14', 'PRD-05', 4, 'Berfungsi dengan baik', '2026-06-15'),
('RVW-12', 'PLG-15', 'PRD-03', 5, 'Pengiriman cepat, packing sangat aman',
'2026-06-15'),
('RVW-13', 'PLG-15', 'PRD-04', 5, 'Pengiriman cepat, packing sangat aman',
'2026-06-15'),
('RVW-14', 'PLG-03', 'PRD-02', 4, 'Kirim ke kantor cepat sampai, sambal
aman', '2026-06-16'),
('RVW-15', 'PLG-10', 'PRD-05', 5, 'Packing tebal, barang aman berguna',
'2026-06-16');
SELECT * FROM REVIEW;
```

ID Review	ID Pelanggan	ID Produk	Rating	Komentar	Tanggal Review
RVW-01	PLG-01	PRD-01	5	Kainnya adem sekali	11/06/2026
RVW-02	PLG-01	PRD-04	5	Serumnya cocok	11/06/2026
RVW-03	PLG-02	PRD-02	4	Sambalnya mantap, cumi tidak alot	10/06/2026

RVW-04	PLG-01	PRD-02	5	Langganan, selalu enak makan pake ini	11/06/2026
RVW-05	PLG-05	PRD-02	3	Rasa oke tapi pengiriman agak lama	12/06/2026
RVW-06	PLG-07	PRD-04	4	Istri saya suka, kerutan berkurang katanya	12/06/2026
RVW-07	PLG-08	PRD-05	5	Sapunya kokoh, gagang mantap	13/06/2026
RVW-08	PLG-10	PRD-01	4	Kemejanya pas	14/06/2026
RVW-09	PLG-10	PRD-03	4	Liptint buat kado adek	14/06/2026
RVW-10	PLG-11	PRD-02	5	Enak banget, pedesnya nampol	14/06/2026
RVW-11	PLG-14	PRD-05	4	Berfungsi dengan baik	15/06/2026
RVW-12	PLG-15	PRD-03	5	Pengiriman cepat, packing sangat aman	15/06/2026

RVW-13	PLG-15	PRD-04	5	Pengiriman cepat, packing sangat aman	15/06/202 6
RVW-14	PLG-03	PRD-02	4	Kirim ke kantor cepat sampai, sambal aman	16/06/202 6
RVW-15	PLG-10	PRD-05	5	Packing tebal, barang aman berguna	16/06/202 6

4.2. Hasil Implementasi Database

Setelah seluruh DDL dan DML dijalankan, berikut adalah hasil verifikasi implementasi basis data marketplace yang mencakup struktur tabel, relasi antar tabel, dan verifikasi data.

4.2.1. Struktur Tabel

Basis data marketplace berhasil dibuat dengan dua belas (12) tabel utama. Tabel berikut menyajikan ringkasan struktur setiap tabel beserta jumlah kolom dan jenis primary key yang digunakan.

No.	Nama Tabel	Primary Key	Foreign Key
1	wilayah	kode_pos	–
2	pelanggan	id_pelanggan	–
3	alamat	id_alamat	id_pelanggan → pelanggan; kode_pos → wilayah
4	kategori	id_kategori	–
5	produk	id_produk	id_kategori → kategori
6	keranjang	id_keranjang	id_pelanggan → pelanggan
7	detail_keranjang	id_detail_keranjang	id_keranjang → keranjang; id_produk → produk

8	pesanan	id_pesanan	id_pelanggan → pelanggan; id_alamat → alamat
9	detail_pesanan	id_detail_pesanan	id_pesanan → pesanan; id_produk → produk
10	pembayaran	id_pembayaran	id_pesanan → pesanan
11	pengiriman	id_pengiriman	id_pesanan → pesanan
12	review	id_review	id_pelanggan → pelanggan; id_produk → produk

Seluruh kolom primary key dan foreign key pada rancangan ini menggunakan tipe data VARCHAR(10) dengan format kode (contoh: PLG-01, PRD-01, ORD-001), kecuali tabel wilayah yang primary key-nya berupa kode_pos asli. Tabel wilayah merupakan hasil dekomposisi 3NF dari tabel alamat dan menjadi tabel referensi pertama yang harus diisi sebelum tabel alamat.

4.3. Relasi Antar Tabel

Relasi antar tabel dalam basis data marketplace membentuk tiga pola hubungan utama sebagai berikut.

- 1) One-to-Many (1:N): Satu wilayah (kode_pos) dapat digunakan oleh banyak alamat. Satu pelanggan dapat memiliki banyak alamat, banyak pesanan, banyak keranjang, dan banyak ulasan (review). Satu kategori dapat memiliki banyak produk. Satu pesanan dapat memiliki banyak detail pesanan, satu pembayaran, dan satu pengiriman.
- 2) Many-to-Many (M:N) melalui tabel pivot: Hubungan antara produk dan keranjang diwakili oleh tabel detail_keranjang. Hubungan antara produk dan pesanan diwakili oleh tabel detail_pesanan.
- 3) One-to-One (1:1) konseptual: Setiap pesanan memiliki tepat satu catatan pembayaran dan satu catatan pengiriman.

Diagram ERD pada Gambar 4.1 menunjukkan keseluruhan relasi tersebut. Integritas referensial dijaga melalui FOREIGN KEY constraint sehingga setiap data pesanan pasti merujuk pada pelanggan dan alamat yang valid, setiap alamat pasti merujuk pada kode_pos yang valid di tabel wilayah, dan setiap detail pesanan pasti merujuk pada pesanan serta produk yang ada di basis data.

4.4. Ringkasan Implementasi

Implementasi basis data marketplace telah berhasil diselesaikan secara menyeluruh, termasuk penerapan normalisasi 3NF melalui pemecahan tabel wilayah. Berikut adalah ringkasan dari seluruh capaian implementasi:

Aspek Implementasi	Capaian
Basis Data	1 database (marketplace1) berhasil dibuat
Jumlah Tabel	12 tabel utama, termasuk tabel wilayah hasil normalisasi 3NF
Primary Key	12 kolom PK bertipe VARCHAR(10) dengan format kode manual
Foreign Key	14 relasi FK yang menjaga integritas referensial
Constraint Lain	UNIQUE (email), CHECK (rating 1-5), DEFAULT ('aktif', FALSE, CURRENT_DATE), NOT NULL
Indeks	12 indeks otomatis (PK/UNIQUE) + 7 indeks manual pada kolom kritis
Data Dummy Wilayah	16 kode pos dari berbagai kota di Indonesia
Data Dummy Pelanggan	15 akun aktif dari berbagai kota di Indonesia
Data Dummy Produk	5 produk dari 4 kategori (Fashion, Kuliner, Kecantikan, Rumah Tangga)
Data Dummy Pesanan	20 pesanan dengan 4 variasi status dan 24 baris detail pesanan
Verifikasi Struktur	Seluruh tabel dan kolom sesuai rancangan 3NF terbaru
Verifikasi Relasi	JOIN lintas tabel (termasuk alamat-wilayah) berhasil menghasilkan data yang konsisten

Secara keseluruhan, implementasi basis data marketplace telah memenuhi kebutuhan fungsional sistem e-commerce sekaligus memenuhi kaidah normalisasi bentuk normal ketiga (3NF). Pemecahan tabel wilayah dari tabel alamat menghilangkan ketergantungan transitif dan mencegah duplikasi data kota/provinsi, sementara penggunaan primary key berformat kode (VARCHAR) menggantikan AUTO_INCREMENT membuat data lebih mudah dibaca dan ditelusuri secara manual. Struktur tabel yang dirancang mampu menangani skenario transaksi dari hulu ke hilir, mulai dari manajemen akun pelanggan dan produk, proses pemesanan dengan

detail item, pengelolaan pembayaran dan pengiriman, hingga sistem ulasan dan penilaian produk

BAB V

PENGUJIAN QUERY

5.1. Pengujian Query SELECT (10 Query Bertingkat)

5.1.1. Query Sederhana (3 Buah Query)

Pada bagian pertama, dilakukan pengujian untuk melihat fungsionalitas pencarian data pada tingkat dasar atau sederhana. Pengujian ini dibagi menjadi tiga bagian kasus sebagai berikut:

1. Kasus pertama ditujukan untuk pemetaan pelanggan berdasarkan domain surel yang digunakan. Tim Manajemen Hubungan Pelanggan (*Customer Relationship Management*) membutuhkan data pengguna yang memanfaatkan layanan Gmail untuk memetakan metode masuk sistem yang paling optimal. Perintah SQL yang dijalankan adalah:

```
-- 1. menampilkan pelanggan yang menggunakan email dari
gmail.com
select id_pelanggan, nama_pelanggan, email
from pelanggan
where email like '%@gmail.com';
```

id_pelanggan	nama_pelanggan	email
PLG-01	Siti Aminah	sitiaminah@gmail.com
PLG-03	Dewi Lestari	dewi.les@gmail.com
PLG-05	Rizky Hidayat	rizky_hid@gmail.com
PLG-06	Fitriani	fitri.ani@gmail.com
PLG-07	Hendra Wijaya	hendraw@gmail.com
PLG-08	Andi Wijaya	andiw99@gmail.com
PLG-09	Mega Utami	mega_utami@gmail.com
PLG-10	Rian Hidayat	rian_hid@gmail.com
PLG-11	Anita Sari	anita.sari@gmail.com
PLG-12	Eko Prasetyo	ekopras@gmail.com
PLG-13	Sri Wahyuni	sri.wahyuni@gmail.com
PLG-14	Bambang T.	bambangt@gmail.com
PLG-15	Indah Permata	indah.per@gmail.com
NULL	NULL	NULL

Gambar 5.1.1.1 Output Kasus Pertama

Berdasarkan hasil tabel pengiriman data pada Gambar 5.1.1.1, dapat dijelaskan bahwa query berhasil menyaring data dari tabel pelanggan dengan kriteria surel yang spesifik. Kolom email pada grid hasil menunjukkan bahwa seluruh akun yang tampil memiliki akhiran '@gmail.com'. Hal ini membuktikan bahwa penggunaan operator LIKE dan *wildcard* persen berfungsi dengan akurat dalam mengisolasi data teks, sehingga tim Manajemen Hubungan Pelanggan mendapatkan data pengguna Gmail yang valid.

2. Kasus kedua ditujukan untuk memantau urutan transaksi masuk berdasarkan tanggal pembuatan nota. Bagian pengemasan dan pemrosesan barang di gudang memerlukan urutan antrean transaksi dari yang paling lama hingga yang terbaru. Langkah ini dilakukan demi menerapkan prinsip *First In, First Out* (FIFO), yaitu metode pengelolaan antrean di mana transaksi yang lebih dulu masuk akan diproses dan diselesaikan pertama kali. Perintah SQL yang dijalankan adalah:

```
-- 2. menampilkan pesanan yang masuk lebih dulu (dari tanggal  
terlama)  
select id_pesanan, tanggal_pesanan, total_harga, status_pesanan  
from pesanan  
order by tanggal_pesanan asc;
```

id_pesanan	tanggal_pesanan	total_harga	status_pesanan
ORD-001	2026-06-10	235000	Selesai
ORD-002	2026-06-10	90000	Selesai
ORD-003	2026-06-11	240000	Diproses
ORD-004	2026-06-11	120000	Menunggu
ORD-005	2026-06-11	45000	Selesai
ORD-006	2026-06-12	135000	Selesai
ORD-007	2026-06-12	150000	Dibatalkan
ORD-008	2026-06-12	170000	Selesai
ORD-009	2026-06-12	80000	Selesai
ORD-010	2026-06-13	180000	Diproses

id_pesanan	tanggal_pesanan	total_harga	status_pesanan
ORD-011	2026-06-13	340000	Selesai
ORD-012	2026-06-13	45000	Selesai
ORD-013	2026-06-14	150000	Diproses
ORD-014	2026-06-14	170000	Menunggu
ORD-015	2026-06-14	40000	Selesai
ORD-016	2026-06-14	175000	Selesai
ORD-017	2026-06-15	150000	Diproses
ORD-018	2026-06-15	45000	Selesai
ORD-019	2026-06-15	135000	Diproses
ORD-020	2026-06-15	80000	Selesai

Gambar 5.1.1.2 Output Kasus Kedua

Berdasarkan hasil tabel pengujian pada Gambar 5.1.1.2, dapat dijelaskan bahwa baris data pada tabel pesanan telah terurut secara kronologis. Kolom tanggal pesanan menampilkan urutan waktu dari tanggal yang paling lampau menuju tanggal yang paling baru. Efek dari klausa `ORDER BY` dengan parameter `ASC` ini berjalan sukses, sehingga pihak gudang dapat melihat dengan jelas nota pesanan tertua yang harus didahulukan proses pengemasannya.

3. Kasus ketiga dirancang untuk menganalisis arus kas masuk terbesar yang berhasil tercatat oleh sistem keuangan pada bagian kasir. Manajemen memerlukan laporan performa transaksi untuk melihat lima transaksi teratas dengan nominal tertinggi yang statusnya valid dan sudah dibayarkan oleh konsumen. Perintah SQL yang dijalankan adalah:

```
-- 3. menampilkan pembayaran yang sudah dibayarkan (≠ 0)
dengan jumlah bayar diurutkan dari yg terbesar
-- hanya menampilkan 5 baris teratas
select id_pembayaran, metode_pembayaran, jumlah_bayar,
tanggal_bayar, status_pembayaran
from pembayaran
where jumlah_bayar <> 0
order by jumlah_bayar desc
limit 5;
```

id_pembayaran	metode_pembayaran	jumlah_bayar	tanggal_bayar	status_pembayaran
PMB-10	Transfer Bank	340000	2026-06-13	Lunas
PMB-03	Transfer Bank	240000	2026-06-11	Lunas
PMB-01	Transfer Bank	235000	2026-06-10	Lunas
PMB-09	E-Wallet (Dana)	180000	2026-06-13	Lunas
PMB-15	Kartu Kredit	175000	2026-06-14	Lunas
NULL	NULL	NULL	NULL	NULL

Gambar 5.1.1.3 Output Kasus Ketiga

Berdasarkan hasil tabel pengujian pada Gambar 5.1.1.3, dapat dijelaskan bahwa sistem berhasil menampilkan lima rekor transaksi dengan nominal pembayaran tertinggi. Kondisi filter `WHERE` terbukti berhasil membuang data pesanan kosong yang bernilai nol. Selain itu, kombinasi klausa `ORDER BY ... DESC` dan `LIMIT 5` berfungsi secara tepat dalam mengurutkan data dari angka terbesar sekaligus mengunci tampilan tabel agar hanya memuat lima baris data teratas.

5.1.2. Query dengan JOIN Minimal 3 Tabel (Memuat Kasus 4 sampai Kasus 7)

Tahap pengujian kedua berfokus pada penggabungan informasi relasional dari beberapa tabel sekaligus untuk menyajikan laporan yang lebih terintegrasi. Implementasi query penggabungan (*JOIN*) ini dijabarkan ke dalam empat studi kasus berikut:

1. Kasus keempat dirancang untuk memenuhi kebutuhan logistik dan pencetakan faktur penjualan (*invoice*). Dalam skenario ini, pihak administrator memerlukan data akurat mengenai alamat tujuan pengiriman barang secara lengkap, termasuk informasi kota dan provinsi asal, khusus untuk semua nota pesanan yang transaksinya telah dinyatakan 'Selesai'. Perintah SQL yang dijalankan adalah:

```
-- 4. Menampilkan alamat pengiriman pesanan lengkap dengan kota
dan provinsi untuk pesanan yang sudah 'Selesai'
SELECT
    p.id_pesanan,
    pl.nama_pelanggan,
    a.alamat_lengkap,
    w.kota,
```

```

w.provinsi
FROM pesanan p
JOIN pelanggan pl ON p.id_pelanggan = pl.id_pelanggan
JOIN alamat a ON p.id_alamat = a.id_alamat
JOIN wilayah w ON a.kode_pos = w.kode_pos
WHERE p.status_pesanan = 'Selesai';

```

id_pesanan	nama_pelanggan	alamat_lengkap	kota	provinsi
ORD-001	Siti Aminah	Jl. Anggrek No. 12, Kel. Merdeka	Bandung	Jawa Barat
ORD-002	Budi Santoso	Perum Gading Indah Blok C/4	Surabaya	Jawa Timur
ORD-005	Siti Aminah	Jl. Anggrek No. 12, Kel. Merdeka	Bandung	Jawa Barat
ORD-006	Rizky Hidayat	Sudirman Central Business District Tbk	Jakarta Selatan	DKI Jakarta
ORD-008	Hendra Wijaya	Jl. Pemuda No. 88, Klitren	Yogyakarta	DI Yogyakarta
ORD-009	Andi Wijaya	Jl. Gejayan No. 10B, Condongcatur	Sleman	DI Yogyakarta
ORD-011	Rian Hidayat	Jl. Merdeka No. 17, Kel. Kidul	Malang	Jawa Timur
ORD-012	Anita Sari	Kost Orange, Gg. Sayur No. 2	Bogor	Jawa Barat
ORD-015	Bambang T.	Jl. Gatot Subroto No. 99	Bandung	Jawa Barat
ORD-016	Indah Permata	Town House cluster Diamond No. 2	Depok	Jawa Barat
ORD-018	Dewi Lestari	Menara Imperium Lt. 12, Kuningan	Jakarta Selatan	DKI Jakarta
ORD-020	Rian Hidayat	Jl. Merdeka No. 17, Kel. Kidul	Malang	Jawa Timur

Gambar 5.1.2.1 Output Kasus Keempat

Berdasarkan hasil tabel pengujian pada Gambar 5.1.2.1, dapat dijelaskan bahwa operasi penggabungan (*JOIN*) lintas empat tabel berjalan dengan sukses tanpa mengalami anomali data. Integrasi data relasional ini dilakukan dengan menghubungkan tabel pesanan, tabel pelanggan, tabel alamat, dan tabel wilayah. Alur relasi pada query ini bekerja dengan menghubungkan tabel pesanan ke tabel pelanggan melalui kolom kunci `id_pelanggan`, sekaligus mengaitkan tabel pesanan ke tabel alamat menggunakan kolom `id_alamat`. Selanjutnya, tabel alamat dihubungkan dengan tabel wilayah melalui kolom kunci `kode_pos`. *Output grid* akhirnya menampilkan kombinasi informasi yang konsisten mulai dari kode pesanan, nama pembeli, hingga detail lokasi *dropping* paket. Hubungan relasional antara tabel alamat dan tabel wilayah yang merupakan hasil pecahan struktur normalisasi bentuk ketiga (3NF) terbukti saling terikat dengan sangat akurat melalui kesamaan data pada kolom kunci tamu (*foreign key*) kode pos tersebut.

2. Kasus kelima dibuat untuk membantu mempermudah kerja bagian layanan pelanggan (*Customer Service*). Petugas membutuhkan data terpadu dalam satu tampilan untuk melacak status pengiriman paket konsumen, mulai dari nama

penerima, nomor telepon, kurir ekspedisi yang membawa barang, hingga nomor resi resmi dari pihak logistik. Perintah SQL yang dijalankan adalah:

```
SELECT
    p.id_pesanan,
    p.tanggal_pesanan,
    a.nama_penerima,
    a.no_hp,
    pg.kurir,
    pg.nomor_resi,
    a.alamat_lengkap,
    pg.status_pengiriman
FROM pesanan p
JOIN alamat a ON p.id_alamat = a.id_alamat
JOIN pengiriman pg ON p.id_pesanan = pg.id_pesanan;
```

id_pesanan	tanggal_pesanan	nama_penerima	no_hp	kurir	nomor_resi	alamat_lengkap
ORD-001	2026-06-10	Siti Aminah	08123456789	J&T Express	JNT100239481	Jl. Anggrek No. 12, Kel. Merdeka
ORD-002	2026-06-10	Budi Santoso	08139876543	SiCepat Anteraja	SCP992834111	Perum Gading Indah Blok C/4
ORD-003	2026-06-11	Rumah Ibu Dewi	08571122334	JNE Express	JNE003928114	Gang Kelinci No. 45, RT 02/RW 05
ORD-005	2026-06-11	Siti Aminah	08123456789	J&T Express	JNT100239499	Jl. Anggrek No. 12, Kel. Merdeka
ORD-006	2026-06-12	Kantor Rizky	08998877665	SiCepat Anteraja	SCP992834150	Sudirman Central Business District Tbk
ORD-008	2026-06-12	Hendra Wijaya	08523344556	JNE Express	JNE003928155	Jl. Pemuda No. 88, Klitren
ORD-009	2026-06-12	Toko Buku Andi	08784455667	J&T Express	JNT100239512	Jl. Gejayan No. 10B, Condongcatur
ORD-010	2026-06-13	Mega Utami	08129988776	SiCepat Anteraja	SCP992834199	Perumahan Bumi Asri Blok F/12

id_pesanan	tanggal_pesanan	nama_penerima	no_hp	kurir	nomor_resi	alamat_lengkap
ORD-011	2026-06-13	Rian Hidayat	08137766554	J&T Express	JNT100239555	Jl. Merdeka No. 17, Kel. Kidul
ORD-012	2026-06-13	Anita Sari	08564433221	Ninja Xpress	NJA882911002	Kost Orange, Gg. Sayur No. 2
ORD-013	2026-06-14	Eko Prasetyo	08126655443	JNE Express	JNE003928210	Jl. Pemuda Selatan No. 14
ORD-015	2026-06-14	Bambang T.	08575544332	J&T Express	JNT100239600	Jl. Gatot Subroto No. 99
ORD-016	2026-06-14	Indah Permata	08221122334	SiCepat Anteraja	SCP992834211	Town House cluster Diamond No. 2
ORD-017	2026-06-15	Budi Santoso	08139876543	JNE Express	JNE003928315	Perum Gading Indah Blok C/4
ORD-018	2026-06-15	Kantor Dewi	08218888999	SiCepat Anteraja	SCP992834300	Menara Imperium Lt. 12, Kuningan
ORD-019	2026-06-15	Ahmad Fauzi	08215566778	J&T Express	JNT100239611	Komplek dosen Unsoed No. B9
ORD-020	2026-06-15	Rian Hidayat	08137766554	J&T Express	JNT100239699	Jl. Merdeka No. 17, Kel. Kidul

Gambar 5.1.2.2 Output Kasus Kelima

Berdasarkan hasil tabel pengujian pada Gambar 5.1.2.2, dapat dijelaskan bahwa operasi penggabungan (*JOIN*) lintas tiga tabel berjalan dengan sukses tanpa ada kendala data. Integrasi ini dilakukan dengan menghubungkan tabel pesanan, tabel alamat, dan tabel pengiriman. Hubungan relasi pada query ini bekerja dengan mengaitkan tabel pesanan ke tabel alamat melalui kecocokan kolom kunci id_alamat, sekaligus menyambungkan tabel pesanan ke tabel pengiriman berdasarkan kesamaan kolom kunci id_pesanan. Hasil pada *output*

grid secara konsisten menyajikan informasi pelacakan paket yang utuh, sehingga petugas layanan pelanggan bisa langsung memantau kurir dan status pengiriman setiap kali ada konsumen yang bertanya.

3. Kasus keenam dirancang untuk membantu tim audit keuangan memeriksa rincian produk apa saja yang sudah terjual berdasarkan kelompok kategorinya, lalu mencocokkannya dengan laporan kas masuk. Agar laporan ini akurat, sistem akan menyaring dan hanya menampilkan item-item belanjaan dari transaksi yang aliran dananya sudah dikonfirmasi berstatus 'Lunas' oleh pihak kasir di sistem pembayaran. Perintah SQL yang dijalankan adalah:

```
-- 6. menampilkan pesanan dengan kategori, produk dan
pembayarannya (hanya menampilkan status pembayaran yang lunas)
SELECT
    k.nama_kategori,
    pr.nama_produk,
    dp.jumlah_beli,
    dp.subtotal AS harga_item,
    pmb.metode_pembayaran,
    pmb.status_pembayaran
FROM detail_pesanan dp
JOIN produk pr ON dp.id_produk = pr.id_produk
JOIN kategori k ON pr.id_kategori = k.id_kategori
JOIN pembayaran pmb ON dp.id_pesanan = pmb.id_pesanan
WHERE pmb.status_pembayaran = 'Lunas';
```

nama_kategori	nama_produk	jumlah_beli	harga_item	metode_pembayaran	status_pembayaran
Fashion	Kemeja Flannel Pria	1	150000	Transfer Bank	Lunas
Fashion	Kemeja Flannel Pria	1	150000	Transfer Bank	Lunas
Fashion	Kemeja Flannel Pria	1	150000	Transfer Bank	Lunas
Fashion	Kemeja Flannel Pria	1	150000	Transfer Bank	Lunas
Fashion	Kemeja Flannel Pria	1	150000	Transfer Bank	Lunas
Kuliner	Sambal Cumi Asin 200g	3	135000	Transfer Bank	Lunas
Kuliner	Sambal Cumi Asin 200g	1	45000	E-Wallet (Dana)	Lunas
Kuliner	Sambal Cumi Asin 200g	1	45000	E-Wallet (OVO)	Lunas
Kuliner	Sambal Cumi Asin 200g	3	135000	Kartu Kredit	Lunas
Kuliner	Sambal Cumi Asin 200g	1	45000	E-Wallet (Gopay)	Lunas
Kuliner	Sambal Cumi Asin 200g	2	90000	E-Wallet (OVO)	Lunas

nama_kategori	nama_produk	jumlah_beli	harga_item	metode_pembayaran	status_pembayaran
Kecantikan	Lip Tint Cherry	1	90000	Kartu Kredit	Lunas
Kecantikan	Lip Tint Cherry	2	180000	Transfer Bank	Lunas
Kecantikan	Lip Tint Cherry	2	180000	E-Wallet (Dana)	Lunas
Kecantikan	Lip Tint Cherry	1	90000	Transfer Bank	Lunas
Kecantikan	Serum Retinol 20ml	1	85000	Kartu Kredit	Lunas
Kecantikan	Serum Retinol 20ml	2	170000	Transfer Bank	Lunas
Kecantikan	Serum Retinol 20ml	1	85000	Transfer Bank	Lunas
Rumah Tangga	Sapu Lantai Rayon	2	80000	E-Wallet (Gopay)	Lunas
Rumah Tangga	Sapu Lantai Rayon	1	40000	E-Wallet (Gopay)	Lunas
Rumah Tangga	Sapu Lantai Rayon	2	80000	E-Wallet (Gopay)	Lunas

Gambar 5.1.2.3 Output Kasus Keenam

Berdasarkan hasil tabel pengujian pada Gambar 5.1.2.3, dapat dijelaskan bahwa operasi penggabungan (*JOIN*) lintas empat tabel berjalan dengan sukses tanpa mengalami kesalahan data. Integrasi data ini dilakukan dengan menghubungkan tabel detail pesanan, tabel produk, tabel kategori, dan tabel pembayaran. Hubungan relasi pada query ini bekerja dengan mengaitkan tabel detail pesanan ke tabel produk melalui kolom kunci *id_produk*, serta menghubungkan tabel produk ke tabel kategori menggunakan kolom *id_kategori*. Sementara itu, tabel detail pesanan disinkronkan dengan tabel pembayaran melalui kolom kunci *id_pesanan*. Filter *WHERE* terbukti berhasil membatasi baris data sehingga *output grid* hanya menampilkan item produk yang transaksinya benar-benar lunas dan sah secara keuangan.

4. Kasus ketujuh dirancang untuk membantu tim pengembang produk (*product development*) dalam memantau tingkat kepuasan konsumen. Tim memerlukan data ulasan tertulis beserta skor penilaian bintang yang dikelompokkan langsung berdasarkan kategori barangnya, agar mereka bisa mengevaluasi jenis produk mana yang kualitas atau fungsinya perlu ditingkatkan. Perintah SQL yang dijalankan adalah:

```
-- 7.Menampilkan ulasan produk (review) yang diberikan oleh
pelanggan lengkap dengan nama kategori produknya
SELECT
    r.id_review,
    pl.nama_pelanggan,
    pr.nama_produk,
    k.nama_kategori,
```



```

    r.rating,
    r.komentar
FROM review r
JOIN pelanggan pl ON r.id_pelanggan = pl.id_pelanggan
JOIN produk pr ON r.id_produk = pr.id_produk
JOIN kategori k ON pr.id_kategori = k.id_kategori;

```

id_review	nama_pelanggan	nama_produk	nama_kategori	rating	komentar
RVW-01	Siti Aminah	Kemeja Flannel Pria	Fashion	5	Kainnya adem sekali
RVW-02	Siti Aminah	Serum Retinol 20ml	Kecantikan	5	serumnya cocok
RVW-03	Budi Santoso	Sambal Cumi Asin 200g	Kuliner	4	Sambalnya mantap, cumi tidak alot
RVW-04	Siti Aminah	Sambal Cumi Asin 200g	Kuliner	5	Langganan, selalu enak makan pake ini
RVW-05	Rizky Hidayat	Sambal Cumi Asin 200g	Kuliner	3	Rasa oke tapi pengiriman agak lama
RVW-06	Hendra Wijaya	Serum Retinol 20ml	Kecantikan	4	Istri saya suka, kerutan berkurang katanya
RVW-07	Andi Wijaya	Sapu Lantai Rayon	Rumah Tangga	5	Sapunya kokoh, gagang mantap
RVW-08	Rian Hidayat	Kemeja Flannel Pria	Fashion	4	Kemejanya pas

id_review	nama_pelanggan	nama_produk	nama_kategori	rating	komentar
RVW-09	Rian Hidayat	Lip Tint Cherry	Kecantikan	4	liptint buat kado adek
RVW-10	Anita Sari	Sambal Cumi Asin 200g	Kuliner	5	Enak banget, pedesnya nampol
RVW-11	Bambang T.	Sapu Lantai Rayon	Rumah Tangga	4	Berfungsi dengan baik
RVW-12	Indah Permata	Lip Tint Cherry	Kecantikan	5	Pengiriman cepat, packing sangat aman
RVW-13	Indah Permata	Serum Retinol 20ml	Kecantikan	5	Pengiriman cepat, packing sangat aman
RVW-14	Dewi Lestari	Sambal Cumi Asin 200g	Kuliner	4	Kirim ke kantor cepat sampai, sambal aman
RVW-15	Rian Hidayat	Sapu Lantai Rayon	Rumah Tangga	5	Packing tebal, barang aman berguna

Gambar 5.1.2.4 Output Kasus Ketujuh

Berdasarkan hasil tabel pengujian pada Gambar 5.1.2.4, dapat dijelaskan bahwa operasi penggabungan (*JOIN*) lintas empat tabel berjalan dengan sukses tanpa ada kendala data. Integrasi data ini dilakukan dengan menghubungkan tabel review, tabel pelanggan, tabel produk, dan tabel kategori. Alur relasi dalam query ini bekerja dengan mengaitkan tabel review ke tabel pelanggan menggunakan kolom kunci *id_pelanggan*, sekaligus menghubungkannya ke tabel produk melalui kolom *id_produk*. Selanjutnya, tabel produk disambungkan dengan tabel kategori menggunakan kolom kunci *id_kategori*. Hasil pada *output grid* secara konsisten menyandingkan nama pelanggan, detail produk, kelompok kategori, hingga angka rating dan ulasan teks secara rapi, sehingga tim pengembang bisa langsung memetakan performa produk berdasarkan respon riil pengguna.

5.1.3. Query dengan Subquery atau CTE

Pada bagian ketiga, dilakukan pengujian untuk melihat fungsionalitas pencarian data pada tingkat lanjut menggunakan *Subquery* (query bersarang) dan *Common Table Expression* (CTE). Pengujian ini dibagi menjadi dua bagian kasus sebagai berikut:

1. Kasus delapan : Menggunakan Subquery untuk Mencari Karyawan dengan Umur di Atas Rata-Rata. Kasus ini bertujuan untuk menampilkan daftar karyawan yang memiliki umur (*age*) lebih tua dibandingkan dengan rata-rata umur seluruh karyawan di dalam perusahaan. Karena rata-rata umur bersifat dinamis, digunakan *scalar subquery* pada klausa *WHERE*. Perintah SQL yang dijalankan adalah:

```
-- 8. Menampilkan daftar produk yang harganya di atas
rata-rata harga seluruh produk (Subquery)
SELECT id_produk, nama_produk, harga_produk, stok
FROM produk
WHERE harga_produk > (SELECT AVG(harga_produk) FROM produk);
```

	id_produk	nama_produk	harga_produk	stok
▶	PRD-01	Kemeja Flannel Pria	150000	45
	PRD-03	Lip Tint Cherry	90000	200
	PRD-04	Serum Retinol 20ml	85000	120
✱	NULL	NULL	NULL	NULL

Gambar 5.1.3.1 Output Kasus Ke delapan

Penjelasan:

- Query di dalam kurung (SELECT AVG(age) FROM employees) berjalan terlebih dahulu untuk menghitung nilai rata-rata umur dari semua baris di tabel employees.
- Nilai rata-rata tersebut kemudian digunakan oleh query utama sebagai standar pembanding pada klausa WHERE age >
- Hasil akhir hanya akan menampilkan karyawan seperti Eko Saputra (35 tahun) atau Andi Wijaya (30 tahun) jika umur mereka berada di atas angka rata-rata tersebut.

Penjelasan:

- WITH RataRataStatus AS (...) mendefinisikan sebuah CTE bernama RataRataStatus. Di dalamnya, MySQL mengelompokkan data berdasarkan current_status dan menghitung rata-rata umurnya (avg_age_group).
- Query utama kemudian melakukan JOIN antara tabel fisik employees dengan tabel virtual RataRataStatus berdasarkan kesamaan kolom current_status.
- Klausa WHERE e.age > r.avg_age_group menyaring data sehingga kita bisa melihat karyawan mana yang "paling senior secara umur" di kelompok status kerjanya masing-masing. Penggunaan CTE membuat query yang rumit ini menjadi jauh lebih rapi dan mudah dibaca dibandingkan menggunakan *nested subquery*.

2. Kasus sembilan : Menggunakan CTE untuk Analisis Karyawan Berdasarkan Rata-Rata Umur per Status Kerja Kasus ini memanfaatkan *Common Table Expression* (CTE) untuk membuat tabel virtual sementara. Tujuannya adalah untuk mengidentifikasi siapa saja karyawan yang umurnya melebihi rata-rata umur di kelompok status kerja mereka masing-masing (misalnya kelompok 'employed' atau 'probation'). Perintah SQL yang dijalankan adalah:

```
-- 9. Menampilkan total belanja per pelanggan menggunakan CTE
(Common Table Expression)
WITH TotalBelanjaPelanggan AS (
    SELECT id_pelanggan, SUM(total_harga) AS total_pengeluaran
    FROM pesanan
    WHERE status_pesanan = 'Selesai'
    GROUP BY id_pelanggan
)
SELECT p.id_pelanggan, p.nama_pelanggan, t.total_pengeluaran
FROM pelanggan p
JOIN TotalBelanjaPelanggan t ON p.id_pelanggan = t.id_pelanggan
ORDER BY t.total_pengeluaran DESC;
```

	id_pelanggan	nama_pelanggan	total_pengeluaran
▶	PLG-10	Rian Hidayat	420000
	PLG-01	Siti Aminah	280000
	PLG-15	Indah Permata	175000
	PLG-07	Hendra Wijaya	170000
	PLG-05	Rizky Hidayat	135000
	PLG-02	Budi Santoso	90000
	PLG-08	Andi Wijaya	80000
	PLG-03	Dewi Lestari	45000
	PLG-11	Anita Sari	45000
	PLG-14	Bambang T.	40000

Gambar 5.1.3.1 Output Kasus Kesembilan

5.1.4. Query Fungsi Agregat & GROUP BY + HAVING

Pada bagian ini dilakukan pengujian terhadap query yang menggunakan fungsi agregat SUM() dan COUNT() serta klausa GROUP BY dan HAVING. Pengujian bertujuan untuk menampilkan kategori produk yang memiliki total pendapatan dari pesanan dengan status 'Selesai' lebih dari Rp200.000, sekaligus menghitung jumlah produk yang terjual pada setiap kategori. Data diperoleh dari hasil penggabungan tabel detail_pesanan, produk, kategori, dan pesanan. Perintah SQL yang dijalankan adalah:

```
-- 10. Menampilkan kategori produk yang memiliki total pendapatan
(subtotal selesai) lebih dari Rp 200.000
SELECT
    k.nama_kategori,
    COUNT(dp.id_produk) AS jumlah_produk_terjual,
    SUM(dp.subtotal) AS total_pendapatan
FROM detail_pesanan dp
JOIN produk pr ON dp.id_produk = pr.id_produk
JOIN kategori k ON pr.id_kategori = k.id_kategori
JOIN pesanan p ON dp.id_pesanan = p.id_pesanan
WHERE p.status_pesanan = 'Selesai'
GROUP BY k.nama_kategori
HAVING total_pendapatan > 200000;
```

Berdasarkan hasil tabel pengujian di atas, dapat dijelaskan bahwa eksekusi kueri agregasi tersebut berhasil mengonsolidasikan data dari berbagai tabel relasional

(detail_pesanan, produk, kategori, pesanan) ke dalam satu kesatuan informasi yang ringkas. Kolom-kolom hasil kalkulasi seperti jumlah_produk_terjual dan total_pendapatan menampilkan nilai yang akurat, di mana sistem menyaring data secara spesifik hanya untuk transaksi yang memiliki indikator status_pesanan = 'Selesai'.

Hal ini membuktikan bahwa penggunaan kueri bersambung JOIN, fungsi kelompok GROUP BY, serta pembatasan kondisi kelompok melalui klausa HAVING total_pendapatan > 200000 berjalan dengan sukses. Melalui implementasi ini, tim Manajemen Produk dapat langsung mengevaluasi kriteria kategori produk yang paling menguntungkan dan memiliki performa finansial tinggi secara efisien.

5.2. Pengujian Logical View (2 Buah VIEW)

Pada bagian kedua, dilakukan pengujian untuk mengimplementasikan dan melihat fungsionalitas *Logical View* pada database penjualan. *View* merupakan tabel virtual yang dibuat berdasarkan query penyeleksian data, berfungsi untuk menyederhanakan kompleksitas kueri multi-tabel bagi pengguna akhir serta meningkatkan keamanan data melalui pembatasan akses langsung ke tabel fisik. Pengujian objek *View* ini dibagi menjadi dua bagian kasus sebagai berikut:

1. Kasus kesepuluh ditujukan untuk pemantauan performa penjualan produk secara menyeluruh dan *real-time*. Tim Manajemen Produk dan Penjualan membutuhkan metrik performa komparatif seperti akumulasi kuantitas produk terjual, total omset yang dihasilkan, serta tingkat kepuasan pelanggan melalui rating untuk menentukan strategi pemasaran dan manajemen perputaran stok yang optimal. Perintah SQL yang dijalankan adalah:

```
-- VIEW 1: Menampilkan ringkasan performa penjualan produk  
secara real-time  
CREATE OR REPLACE VIEW view_performa_produk AS  
SELECT  
    p.id_produk,  
    p.nama_produk,  
    p.harga_produk,  
    p.stok AS sisa_stok,  
    IFNULL(SUM(dp.jumlah_beli), 0) AS total_terjual,
```

```

        IFNULL(SUM(dp.subtotal), 0) AS total_omset,
        IFNULL(ROUND(AVG(r.rating), 1), 0) AS rata_rata_rating
FROM produk p
LEFT JOIN detail_pesanan dp ON p.id_produk = dp.id_produk
LEFT JOIN pesanan ps ON dp.id_pesanan = ps.id_pesanan AND
ps.status_pesanan = 'Selesai'
LEFT JOIN review r ON p.id_produk = r.id_produk
GROUP BY p.id_produk, p.nama_produk, p.harga_produk, p.stok;

SELECT * from view_performa_produk;

```

	id_produk	nama_produk	harga_produk	sisa_stok	total_terjual	total_omset	rata_rata_rating
▶	PRD-01	Kemeja Flannel Pria	150000	45	12	1800000	4.5
	PRD-02	Sambal Cumi Asin 200g	45000	80	55	2475000	4.2
	PRD-03	Lip Tint Cherry	90000	200	12	1080000	4.5
	PRD-04	Serum Retinol 20ml	85000	120	18	1530000	4.7
	PRD-05	Sapu Lantai Rayon	40000	50	24	960000	4.7

Gambar 5.2.1 Output Kasus Kesepuluh

Berdasarkan hasil tabel pengujian di atas, dapat dijelaskan bahwa pembuatan objek Logical View berhasil mengonsolidasikan data dari berbagai tabel relasional (produk, detail_pesanan, pesanan, dan review) ke dalam satu entitas virtual yang ringkas. Kolom-kolom hasil agregasi seperti total_terjual, total_omset, dan rata_rata_rating menampilkan nilai kalkulasi yang akurat, di mana produk yang belum memiliki transaksi atau ulasan tetap terdata dengan nilai 0 berkat penanganan fungsi IFNULL(). Hal ini membuktikan bahwa penggunaan kueri bersambung LEFT JOIN dan fungsi kelompok GROUP BY di dalam view berjalan dengan sukses, sehingga tim Manajemen Produk dapat mengevaluasi popularitas setiap komoditas tanpa perlu menyusun kueri rumit secara berulang.

2. Kasus kesebelas ditujukan untuk transparansi dan pelacakan status transaksi pelanggan secara *end-to-end*. Tim Layanan Pelanggan (*Customer Service*) dan operasional logistik memerlukan ringkasan informasi yang mengintegrasikan identitas pemesan dengan kejelasan status pembayaran serta pengirimannya demi mempercepat respons aduan pelanggan serta memitigasi kendala keterlambatan interaksi kurir. Perintah SQL yang dijalankan adalah:

```
-- VIEW 2: Menampilkan ringkasan pesanan pelanggan beserta
status pembayaran dan pengirimannya
CREATE OR REPLACE VIEW view_ringkasan_transaksi AS
SELECT
    p.id_pesanan,
    pl.nama_pelanggan,
    p.tanggal_pesanan,
    p.total_harga,
    p.status_pesanan,
    IFNULL(pmb.metode_pembayaran, 'Belum Pilih') AS
metode_bayar,
    IFNULL(pmb.status_pembayaran, 'Menunggu') AS status_bayar,
    IFNULL(pg.kurir, 'Belum Diproses') AS kurir_pengirim,
    IFNULL(pg.status_pengiriman, 'Belum Dikirim') AS
status_kirim
FROM pesanan p
JOIN pelanggan pl ON p.id_pelanggan = pl.id_pelanggan
LEFT JOIN pembayaran pmb ON p.id_pesanan = pmb.id_pesanan
LEFT JOIN pengiriman pg ON p.id_pesanan = pg.id_pesanan;

SELECT * from view_ringkasan_transaksi;
```

id_pesanan	nama_pelanggan	tanggal_pesanan	total_harga	status_pesanan	metode_bayar	status_bayar	kurir_pengirim	status_kirim
ORD-001	Siti Aminah	2026-06-10	235000	Selesai	Transfer Bank	Lunas	J&T Express	Diterima
ORD-005	Siti Aminah	2026-06-11	45000	Selesai	E-Wallet (Gopay)	Lunas	J&T Express	Diterima
ORD-002	Budi Santoso	2026-06-10	90000	Selesai	E-Wallet (OVO)	Lunas	SiCepat Anteraja	Diterima
ORD-017	Budi Santoso	2026-06-15	150000	Diproses	Transfer Bank	Lunas	JNE Express	Dikirim
ORD-003	Dewi Lestari	2026-06-11	240000	Diproses	Transfer Bank	Lunas	JNE Express	Dikirim
ORD-018	Dewi Lestari	2026-06-15	45000	Selesai	E-Wallet (Dana)	Lunas	SiCepat Anteraja	Diterima
ORD-004	Ahmad Fauzi	2026-06-11	120000	Menunggu	Transfer Bank	Menunggu	Belum Diproses	Belum Dikirim
ORD-019	Ahmad Fauzi	2026-06-15	135000	Diproses	Transfer Bank	Lunas	J&T Express	Dikirim
ORD-006	Rizky Hidayat	2026-06-12	135000	Selesai	Kartu Kredit	Lunas	SiCepat Anteraja	Diterima
ORD-007	Fitriani	2026-06-12	150000	Dibatalkan	Belum Pilih	Menunggu	Belum Diproses	Belum Dikirim
ORD-008	Hendra Wijaya	2026-06-12	170000	Selesai	Transfer Bank	Lunas	JNE Express	Diterima
ORD-009	Andi Wijaya	2026-06-12	80000	Selesai	E-Wallet (Gopay)	Lunas	J&T Express	Diterima
ORD-010	Mega Utami	2026-06-13	180000	Diproses	E-Wallet (Dana)	Lunas	SiCepat Anteraja	Dikirim
ORD-011	Rian Hidayat	2026-06-13	340000	Selesai	Transfer Bank	Lunas	J&T Express	Diterima
ORD-020	Rian Hidayat	2026-06-15	80000	Selesai	E-Wallet (Gopay)	Lunas	J&T Express	Diterima
ORD-012	Anita Sari	2026-06-13	45000	Selesai	E-Wallet (OVO)	Lunas	Ninja Xpress	Diterima
ORD-013	Eko Prasetyo	2026-06-14	150000	Diproses	Transfer Bank	Lunas	JNE Express	Dikirim
ORD-014	Sri Wahyuni	2026-06-14	170000	Menunggu	Transfer Bank	Menunggu	Belum Diproses	Belum Dikirim
ORD-015	Bambang T.	2026-06-14	40000	Selesai	E-Wallet (Gopay)	Lunas	J&T Express	Diterima
ORD-016	Indah Permata	2026-06-14	175000	Selesai	Kartu Kredit	Lunas	SiCepat Anteraja	Diterima

Logical View kedua berhasil menyajikan rekapitulasi alur transaksi finansial dan logistik setiap nota secara terpadu. Kolom seperti metode_bayar, status_bayar, kurir_pengirim, dan status_kirim menampilkan data pelacakan yang informatif, di mana nilai default tekstual (seperti 'Belum Pilih', 'Menunggu', atau 'Belum Dikirim') berhasil dimunculkan melalui fungsi

IFNULL() untuk merepresentasikan transaksi baru yang belum menyelesaikan tahapan pemrosesan tertentu. Efek penggabungan tabel menggunakan parameter JOIN dan LEFT JOIN ini berjalan dengan sukses, sehingga pihak operasional mendapatkan sebuah dasbor ringkasan transaksi yang valid dan siap pakai untuk menjaga efisiensi layanan harian.

5.3. Pengujian Stored Procedure (1 Buah)

Pada bagian ini dilakukan pengujian terhadap *stored procedure* yang digunakan untuk menghasilkan laporan pelanggan berdasarkan provinsi. Pengujian bertujuan untuk memastikan bahwa prosedur dapat menggabungkan data dari beberapa tabel serta menghitung jumlah transaksi dan total belanja pelanggan secara otomatis. Pada pengujian ini digunakan data pelanggan yang berasal dari Provinsi Jawa Tengah. Perintah SQL yang dijalankan adalah:

```
-- 1 Buah STORED PROCEDURE
DELIMITER //

CREATE PROCEDURE sp_laporan_pelanggan_by_provinsi(
    IN p_nama_provinsi VARCHAR(100) -- Parameter input provinsi
    sesuai panjang kolom Anda
)
BEGIN
    SELECT
        p.id_pelanggan AS ID_Pelanggan,
        p.nama_pelanggan AS Nama_Pelanggan,
        w.provinsi AS Provinsi,
        COUNT(ps.id_pesanan) AS Jumlah_Transaksi,
        SUM(ps.total_harga) AS Total_Belanja
    FROM pelanggan p
    INNER JOIN alamat a ON p.id_pelanggan = a.id_pelanggan
    INNER JOIN wilayah w ON a.kode_pos = w.kode_pos
    INNER JOIN pesanan ps ON p.id_pelanggan = ps.id_pelanggan
    WHERE w.provinsi = p_nama_provinsi
    GROUP BY p.id_pelanggan, p.nama_pelanggan, w.provinsi
    ORDER BY Total_Belanja DESC;
END //

DELIMITER ;

CALL sp_laporan_pelanggan_by_provinsi('Jawa Tengah');
```


	ID_Pelanggan	Nama_Pelanggan	Provinsi	Jumlah_Transaksi	Total_Belanja
▶	PLG-04	Ahmad Fauzi	Jawa Tengah	2	255000
	PLG-12	Eko Prasetyo	Jawa Tengah	1	150000

Gambar 5.3.1 Output Pengujian Stored Procedure

Berdasarkan hasil pengujian, *stored procedure* berhasil menampilkan data pelanggan yang berasal dari Provinsi Jawa Tengah. Hasil yang diperoleh menunjukkan terdapat dua pelanggan yang memenuhi kriteria, yaitu Ahmad Fauzi dengan 2 transaksi dan total belanja sebesar Rp255.000 serta Eko Prasetyo dengan 1 transaksi dan total belanja sebesar Rp150.000.

Selain itu, data berhasil diurutkan berdasarkan nilai Total_Belanja secara menurun (*descending*) sehingga pelanggan dengan nilai pembelian tertinggi ditampilkan terlebih dahulu. Hal ini menunjukkan bahwa proses *JOIN*, fungsi agregasi *COUNT()* dan *SUM()*, serta pengurutan data menggunakan *ORDER BY DESC* telah berjalan dengan baik. Dengan demikian, *stored procedure* mampu menghasilkan laporan pelanggan berdasarkan provinsi secara akurat sesuai kebutuhan sistem.

5.4. Pengujian Database Trigger (1 Buah)

Pengujian ini dilakukan untuk memastikan bahwa *trigger* dapat bekerja secara otomatis dalam mencatat aktivitas perubahan kata sandi pelanggan ke dalam tabel log keamanan. Pencatatan tersebut bertujuan untuk mendukung proses audit dan meningkatkan keamanan data pengguna. Kasus pengujian dilakukan dengan mengubah nilai kata sandi pada data pelanggan dengan ID PLG-01. Perintah SQL yang dijalankan adalah.

```
-- 1 Buah TRIGGER
CREATE TABLE log_keamanan (
    id_log INT AUTO_INCREMENT PRIMARY KEY,
    id_pelanggan VARCHAR(10),
    aksi VARCHAR(50),
    tanggal_kejadian DATETIME DEFAULT NOW()
);

DELIMITER $$
```

```

CREATE TRIGGER trg_audit_perubahan_password
AFTER UPDATE ON pelanggan
FOR EACH ROW
BEGIN
    -- Logika: Hanya mencatat JIKA kolom password mengalami perubahan
    IF OLD.password <> NEW.password THEN
        INSERT INTO log_keamanan (id_pelanggan, aksi)
        VALUES (NEW.id_pelanggan, 'Pelanggan mengganti password
akun');
    END IF;
END$$

DELIMITER ;

UPDATE pelanggan SET password = 'password_baru_123' WHERE
id_pelanggan = 'PLG-01';
SELECT * FROM log_keamanan;

```

	id_log	id_pelanggan	aksi	tanggal_kejadian
▶	1	PLG-01	Pelanggan mengganti password akun	2026-06-22 17:11:29
•	NULL	NULL	NULL	NULL

Gambar 5.4.1 Output Pengujian Database Trigger

Berdasarkan hasil pengujian yang diperoleh, sistem berhasil menambahkan data baru ke dalam tabel log_keamanan setelah terjadi perubahan kata sandi pada tabel pelanggan. Data yang tercatat pada tabel log keamanan menunjukkan informasi mengenai pelanggan yang melakukan perubahan kata sandi, jenis aktivitas yang dilakukan, serta waktu terjadinya perubahan. asil tersebut membuktikan bahwa *trigger* trg_audit_perubahan_password berhasil mendeteksi perubahan data pada kolom password dan menjalankan proses pencatatan log secara otomatis. Dengan demikian, *trigger* yang diuji telah berfungsi sesuai dengan rancangan sistem dan dapat digunakan untuk mendukung keamanan serta pengawasan aktivitas pengguna.

5.5. Pengujian User-Defined Function (1 Buah)

Pengujian ini dilakukan untuk memastikan bahwa *user-defined function* dapat menentukan tingkatan loyalitas pelanggan berdasarkan total nilai transaksi yang telah

diselesaikan. Fungsi ini digunakan untuk membantu perusahaan dalam mengelompokkan pelanggan sesuai tingkat aktivitas belanjanya. Kasus pengujian dilakukan dengan menampilkan data pelanggan beserta kategori keanggotaan yang dihasilkan oleh fungsi `fn_cek_loyalty_pelanggan`. Perintah SQL yang dijalankan adalah:

```
-- 1 Buah FUNCTION
DELIMITER $$

CREATE FUNCTION fn_cek_loyalty_pelanggan (p_id_pelanggan VARCHAR(20))
RETURNS VARCHAR(50)
DETERMINISTIC
BEGIN
    DECLARE v_total_belanja INT DEFAULT 0;
    DECLARE v_status_loyalty VARCHAR(50);

    -- Menghitung total belanja pelanggan yang sukses (Selesai)
    SELECT IFNULL(SUM(total_harga), 0) INTO v_total_belanja
    FROM pesanan
    WHERE id_pelanggan = p_id_pelanggan AND status_pesanan =
        'Selesai';

    -- Menentukan kategori tingkatan berdasarkan nominal belanja
    IF v_total_belanja >= 500000 THEN
        SET v_status_loyalty = 'Platinum Member';
    ELSEIF v_total_belanja >= 200000 THEN
        SET v_status_loyalty = 'Gold Member';
    ELSEIF v_total_belanja > 0 THEN
        SET v_status_loyalty = 'Silver Member';
    ELSE
        SET v_status_loyalty = 'New Member';
    END IF;

    RETURN v_status_loyalty;
END$$

DELIMITER ;

SELECT          id_pelanggan,          nama_pelanggan,
                fn_cek_loyalty_pelanggan(id_pelanggan) AS tingkatan_member
FROM pelanggan;
```

	id_pelanggan	nama_pelanggan	tingkatan_member
▶	PLG-01	Siti Aminah	Gold Member
	PLG-02	Budi Santoso	Silver Member
	PLG-03	Dewi Lestari	Silver Member
	PLG-04	Ahmad Fauzi	New Member
	PLG-05	Rizky Hidayat	Silver Member
	PLG-06	Fitriani	New Member
	PLG-07	Hendra Wijaya	Silver Member
	PLG-08	Andi Wijaya	Silver Member
	PLG-09	Mega Utami	New Member
	PLG-10	Rian Hidayat	Gold Member
	PLG-11	Anita Sari	Silver Member
	PLG-12	Eko Prasetyo	New Member
	PLG-13	Sri Wahyuni	New Member
	PLG-14	Bambang T.	Silver Member
	PLG-15	Indah Permata	Silver Member

Gambar 5.5.1 Output Pengujian User-Defined Function

Berdasarkan hasil pengujian yang diperoleh, fungsi berhasil menampilkan tingkatan loyalitas untuk setiap pelanggan berdasarkan total nilai transaksi dengan status Selesai. Tingkatan keanggotaan yang dihasilkan terdiri dari New Member, Silver Member, Gold Member, dan Platinum Member sesuai dengan batas nominal belanja yang telah ditentukan pada fungsi. Hasil tersebut menunjukkan bahwa fungsi `fn_cek_loyalty_pelanggan` berhasil melakukan perhitungan total belanja pelanggan menggunakan fungsi agregasi `SUM()` dan menentukan kategori keanggotaan menggunakan struktur kondisi `IF`. Dengan demikian, *user-defined function* yang diuji telah berjalan sesuai dengan rancangan sistem dan mampu mengelompokkan pelanggan berdasarkan tingkat loyalitasnya secara otomatis.

BAB VI

KESIMPULAN DAN SARAN

6.1. Kesimpulan

Berdasarkan hasil perancangan dan implementasi sistem basis data *marketplace* yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Identifikasi Kebutuhan Data dan Perancangan Skema Relasional telah berhasil dilakukan dengan mengidentifikasi 12 entitas utama yang saling terintegrasi, yaitu: Pelanggan, Alamat, Wilayah, Kategori, Produk, Keranjang, Detail Keranjang, Pesanan, Detail Pesanan, Pembayaran, Pengiriman, dan Review. Seluruh entitas tersebut telah dipetakan ke dalam *Entity Relationship Diagram* (ERD) dengan notasi *Crow's Foot* yang menggambarkan relasi antar entitas secara jelas, sehingga mampu mengelola berbagai variasi kategori produk secara terintegrasi dan mendukung alur bisnis *e-commerce* dari hulu ke hilir.
2. Proses Normalisasi Data telah diterapkan secara sistematis mulai dari bentuk *Unnormalized Form* (UNF), *First Normal Form* (1NF), *Second Normal Form* (2NF), hingga *Third Normal Form* (3NF). Hasil normalisasi berhasil mengeliminasi redundansi data dan anomali *insert*, *update*, serta *delete*. Salah satu bukti implementasi 3NF adalah pemecahan tabel Alamat menjadi dua tabel terpisah (Alamat dan Wilayah) untuk menghilangkan ketergantungan transitif antara *kode_pos* dengan *kota* dan *provinsi*, sehingga perubahan data wilayah hanya perlu dilakukan di satu tempat dan konsistensi data tetap terjaga.
3. Implementasi Objek Basis Data Tingkat Lanjut telah berhasil dijalankan dengan baik, meliputi:

- a. 10 Query SELECT bertingkat yang mencakup query sederhana, JOIN minimal 3 tabel, Subquery/CTE, serta fungsi agregat dengan GROUP BY dan HAVING. Seluruh query berhasil menghasilkan output yang sesuai dengan kebutuhan fungsional sistem.
 - b. 2 Logical View (*view_performa_produk* dan *view_ringkasan_transaksi*) yang berhasil menyajikan ringkasan data performa produk dan status transaksi secara terpadu, sehingga memudahkan pengguna dalam mengakses informasi tanpa harus menulis query kompleks secara berulang.
 - c. 1 Stored Procedure (*sp_laporan_pelanggan_by_provinsi*) yang berhasil menghasilkan laporan pelanggan berdasarkan provinsi dengan akumulasi jumlah transaksi dan total belanja secara otomatis.
 - d. 1 Trigger (*trg_audit_perubahan_password*) yang berhasil mencatat setiap perubahan kata sandi pelanggan ke dalam tabel log keamanan secara otomatis, mendukung aspek keamanan dan audit sistem.
 - e. 1 User-Defined Function (*fn_cek_loyalty_pelanggan*) yang berhasil mengklasifikasikan tingkat loyalitas pelanggan (New Member, Silver Member, Gold Member, Platinum Member) berdasarkan total nilai transaksi yang telah diselesaikan.
1. Struktur Basis Data yang Dibangun telah terbukti mampu mendukung operasional platform *marketplace* secara menyeluruh, mulai dari manajemen akun pengguna dan alamat, pengelolaan katalog produk dengan kategori, mekanisme transaksi penjualan melalui keranjang dan pesanan, sistem pembayaran dan pengiriman, hingga fasilitas ulasan pelanggan. Penggunaan *primary key* berformat kode (VARCHAR) dengan pola tertentu (misal: PLG-01, PRD-01, ORD-001) memudahkan pembacaan dan penelusuran data secara manual oleh administrator.

2. Integritas Data telah terjaga dengan baik melalui penerapan *constraint* pada setiap tabel, meliputi 12 *Primary Key*, 14 *Foreign Key*, *UNIQUE constraint* pada kolom email, *CHECK constraint* pada kolom rating (1-5), serta *DEFAULT* dan *NOT NULL* pada kolom-kolom kritis. Seluruh *constraint* berfungsi dengan baik dalam menjaga konsistensi dan validitas data di dalam sistem.

6.2. Saran

Berdasarkan hasil perancangan dan implementasi yang telah dilakukan, terdapat beberapa saran untuk pengembangan lebih lanjut:

1. Penambahan Fitur Keamanan Data yang Lebih Komprehensif: Disarankan untuk menambahkan fitur enkripsi pada data sensitif seperti kata sandi pelanggan menggunakan algoritma hashing yang lebih kuat (misalnya bcrypt atau Argon2) dan implementasi mekanisme *audit trail* yang lebih detail, tidak hanya pada perubahan kata sandi tetapi juga pada aktivitas lain seperti perubahan data produk, pembaruan status pesanan, dan akses data oleh administrator.
2. Optimasi Kinerja Query pada Tabel Berukuran Besar: Seiring bertambahnya volume data transaksi, disarankan untuk melakukan optimasi kinerja melalui pembuatan indeks tambahan pada kolom-kolom yang sering digunakan dalam operasi pencarian dan pengurutan, seperti kombinasi kolom *tanggal_pesanan* dan *status_pesanan* pada tabel Pesanan, serta kolom *tanggal_review* pada tabel Review.
3. Pengembangan Dashboard Analitik berbasis *View* dan *Stored Procedure*: Disarankan untuk menambahkan lebih banyak *view* dan *stored procedure* yang mendukung kebutuhan analitik bisnis, seperti laporan penjualan per periode waktu, analisis produk terlaris (*best-seller*) dengan metode *Pareto* (80/20), analisis churn pelanggan, serta peramalan permintaan produk (*demand forecasting*) menggunakan data historis transaksi.

4. Implementasi Trigger untuk Otomatisasi Bisnis Lainnya: Selain *trigger* audit perubahan kata sandi, disarankan untuk menambahkan *trigger* lain yang mendukung otomatisasi proses bisnis, seperti:
 - a. *Trigger* yang secara otomatis mengubah status produk menjadi "Habis" ketika stok mencapai 0.
 - b. *Trigger* yang mencegah penghapusan data pelanggan yang masih memiliki pesanan aktif.
 - c. *Trigger* yang secara otomatis memperbarui status pesanan menjadi "Dibatalkan" jika pembayaran tidak dikonfirmasi dalam batas waktu tertentu.
5. Pengembangan Antarmuka Pengguna (User Interface) yang Terintegrasi: Basis data yang telah dibangun ini akan lebih bermanfaat jika dilengkapi dengan antarmuka pengguna berbasis *web* atau *mobile* yang memungkinkan interaksi langsung dengan sistem, baik untuk pelanggan (melakukan pemesanan, pembayaran, dan ulasan) maupun untuk administrator (mengelola produk, memantau transaksi, dan menghasilkan laporan).
6. Penambahan Entitas dan Relasi untuk Mendukung Fitur Marketplace yang Lebih Kompleks: Disarankan untuk menambahkan entitas baru seperti:
 - a. Penjual/Seller untuk mendukung model bisnis *multi-vendor* di mana setiap produk dimiliki oleh penjual yang berbeda.
 - b. Voucher/Promo untuk mendukung mekanisme diskon dan promosi penjualan.
 - c. Metode Pengiriman untuk menyimpan berbagai pilihan layanan ekspedisi beserta estimasi biaya dan waktu pengiriman.
 - d. Riwayat Status Pesanan untuk melacak perubahan status pesanan secara kronologis sebagai bahan audit.

7. Uji Coba Skalabilitas dengan Data Volume Besar: Disarankan untuk melakukan pengujian beban (*load testing*) dengan data berjumlah besar (minimal 1 juta baris per tabel) untuk mengukur performa sistem dan mengidentifikasi potensi *bottleneck* yang perlu dioptimalkan, terutama pada operasi JOIN dan agregasi data.
8. Dokumentasi Teknis yang Lebih Lengkap: Disarankan untuk menyusun dokumentasi teknis yang mencakup diagram *Entity Relationship Diagram* (ERD) dalam bentuk digital interaktif, *data dictionary* yang dilengkapi contoh nilai, serta panduan penggunaan *stored procedure*, *function*, dan *trigger* bagi pengembang sistem selanjutnya.

Dengan menerapkan saran-saran di atas, diharapkan sistem basis data *marketplace* yang telah dibangun dapat terus berkembang menjadi fondasi yang lebih kokoh, aman, dan adaptif terhadap dinamika bisnis perdagangan elektronik di masa mendatang.

LAMPIRAN

Lampiran 1. Link AI

<https://share.gemini.google/IqVrgMiJHg2R>

Lampiran 2. Repository Github

<https://github.com/Fatihaturrohman/Projek-Basis-Data-Kelompok-G>

Lampiran 3. Link Data Normalisasi dan Video

 Project Basis Data